Universidad Politécnica
de Madrid

**Escuela Técnica Superior de
Ingenieros Informáticos**

Master in Data Science

Master Thesis

Assigning Probabilities and Utilities in Influence Diagrams Using EDAs

Author: Sergio Beamonte González
Supervisor: Juan Antonio Fernández del Pozo De Salamanca

Madrid, Julio, 2026

This Master Thesis has been deposited in ETSI Informáticos de la Universidad Politécnica de Madrid.

Master Thesis
Master in Data Science

Title: Assigning Probabilities and Utilities in Influence Diagrams Using EDAs
July, 2026

Author: Sergio Beamonte González
Supervisor: Juan Antonio Fernández del Pozo De Salamanca
Artificial Intelligence Department
ETSI Informáticos
Universidad Politécnica de Madrid

Abstract

Influence Diagrams are a standard formalism for decision-making under uncertainty, but specifying their numerical parameters, the conditional probability tables and the utilities, demands a costly and exhaustive elicitation from domain experts, which is the main obstacle to their practical use.

This thesis investigates whether those parameters can instead be recovered automatically from a small set of expert decision rules, assuming the structure of the diagram is already known. The task is formulated as a constrained black-box optimisation and addressed with Estimation of Distribution Algorithms: the optimiser searches over an unconstrained real vector that a decoder always maps to a valid model, and each candidate is scored by how faithfully the policy it induces reproduces the given rules. A proof-of-concept system was built on the SMILE and EDASPY libraries and evaluated on two clinical diagrams of different size, comparing five loss functions and four EDA variants.

The results show that recovery is feasible and improves as more rules are added, but that reproducing the training rules does not guarantee recovering the full policy: the problem is degenerate. The choice of optimiser is the dominant factor, and expressive variants that scale to the larger diagram are the limiting resource. The information carried by the rules resides in the utilities rather than in the probabilities, so recovering only the utilities is both cheaper and more accurate. Finally, the composition of the rule set matters as much as its size, and the most informative rules are the decisive ones. The work provides a working recovery method, a characterisation of its limits, and a set of concrete directions for future research.

Keywords: Influence Diagrams; parameter elicitation; decision rules; Estimation of Distribution Algorithms; black-box optimisation; decision support systems; utility recovery.

Resumen

Los *Diagramas de Influencia* (DIs) son un formalismo gráfico compacto para la toma de decisiones bajo incertidumbre: un único grafo codifica las variables inciertas, las opciones disponibles para el agente y las preferencias sobre los resultados. Su principal obstáculo práctico no es el grafo, sino sus números. Rellenar cada tabla de probabilidad condicional (TPC) y cada valor de utilidad exige largas entrevistas con expertos escasos, y el esfuerzo crece tan rápido con el tamaño del modelo que se convierte en el coste dominante de desplegar un DI en dominios reales como la ayuda a la decisión clínica.

Esta tesis estudia una alternativa a esta elicitación exhaustiva. Suponemos que la estructura del DI ya está dada y nos preguntamos si sus parámetros numéricos pueden *recuperarse automáticamente* a partir de un pequeño conjunto de *reglas de decisión* de la forma "si se cumplen estas condiciones, toma esta acción". Las reglas están mucho más cerca de la forma en que razonan los expertos que las probabilidades en bruto, y por lo general solo se necesitan unas pocas para describir una política. El problema se plantea como una optimización de caja negra con restricciones sobre el espacio conjunto de TPCs y utilidades, y se resuelve con *Algoritmos de Estimación de Distribuciones* (EDAs), una familia de optimizadores evolutivos que guían la búsqueda con un modelo probabilístico de las mejores soluciones encontradas hasta el momento.

El proceso de recuperación se implementó sobre el motor de inferencia SMILE y la biblioteca EDASPY. El optimizador busca sobre un vector real sin restricciones que un decodificador convierte siempre en un modelo válido (softmax para las tablas de probabilidad, sigmoide para las utilidades), de modo que ningún candidato tiene que repararse ni descartarse. Cada candidato se puntúa según lo bien que la política que induce reproduce las reglas del experto, empleando una de cinco funciones de pérdida, y se comparan cuatro variantes de EDA de expresividad creciente (UMDA, EMNA, EGNA y KEDA). El método se evalúa sobre dos diagramas clínicos de distinto tamaño: el pequeño *bypass* (manejo de la cardiopatía coronaria) y el mayor *NHLy* (manejo del linfoma no Hodgkin).

El estudio experimental arroja cinco hallazgos principales. Primero, la recuperación es *factible*: mostrar solo una fracción de las reglas eleva la política recuperada muy por encima del azar, y la precisión crece a medida que se añaden más reglas. Segundo, y de forma central, reproducir las reglas de entrenamiento *no* es lo mismo que recuperar la política: muchos vectores de parámetros satisfacen las reglas mostradas pero discrepan en las no vistas, por lo que el problema está parcialmente condicionado. Tercero, el *optimizador* es el factor dominante: KEDA es el más potente en la red pequeña pero escala mal, de modo que en el diagrama mayor solo los más

económicos EMNA y UMDA resultan asequibles, con EMNA claramente por delante. Cuarto, lo que las reglas realmente determinan son las *utilidades*, no las probabilidades: buscar solo las utilidades reproduce la política casi a la perfección, mientras que añadir las tablas de probabilidad a la búsqueda resulta incluso perjudicial. Quinto, *qué* reglas se muestran importa tanto como cuántas: para un número fijo de reglas, la elección genera una amplia dispersión en la precisión, y las reglas más informativas son las decisivas, aquellas con la mayor diferencia de utilidad entre la acción recomendada y su alternativa.

En resumen, el cuello de botella del método es la degeneración del problema y la expresividad de la búsqueda. La receta práctica que se deriva consiste en recuperar las utilidades manteniendo fijas las probabilidades, combinar un optimizador expresivo con una pérdida sencilla e invertir el esfuerzo del experto en un pequeño conjunto de reglas decisivas. El trabajo entrega una prueba de concepto validada y señala la selección activa de reglas, la explotación de la población degenerada y el escalado a diagramas mayores como las continuaciones más prometedoras.

Contents

Chapter 1

Introduction

This introductory chapter establishes the context, motivation, and core objectives of the thesis. It presents Influence Diagrams as the foundational framework for decision support systems and identifies the exhaustive parameter elicitation process as their primary practical bottleneck. To address this limitation, the chapter outlines the main proposal of this research: automating the recovery of quantitative parameters from a reduced set of expert decision rules using evolutionary optimization. Finally, it defines the scope of the study and provides a structural outline of the subsequent chapters.

1.1 Problem Definition

A *Decision Support System* (DSS) is an interactive computational tool designed to assist a decision-maker in the analysis of unstructured problems by combining data, formal models and analytical techniques [24]. Within this broad family, *Influence Diagrams* (IDs) [15, 17, 29] have become the de-facto standard for representing decision problems under uncertainty and offer a particularly compact graphical formalism: a single directed acyclic graph encodes simultaneously the probabilistic structure of the problem (chance nodes), the controllable choices of the agent (decision nodes) and their preferences over outcomes (utility nodes). Unlike a fully expanded decision tree, an ID allows the analyst to reason about the problem at the level of variables and to compute the optimal policy by direct evaluation of the graph using exact algorithms such as the one of Shachter [27].

The expressive power of an ID rests on two distinct ingredients. The first is its *qualitative structure*, the set of nodes and arcs, which captures the conditional dependencies of the problem. The second is its *quantitative parameterisation*, consisting of the *conditional probability tables* (CPTs) of all chance nodes and the numerical entries of the *utility function*. While the structure can often be elicited from a domain expert in a relatively small number of modelling sessions, the parameterisation grows exponentially with the connectivity of the graph and becomes the real bottleneck of any realistic ID-based DSS [3]. A well-known illustration is the neonatal-jaundice ID of Gómez et al. [12]: more than 8 000 probability entries and over a hundred utility values had to be elicited from clinicians before the model could be deployed, and the very same effort is documented as the dominant cost of the project in the companion analysis of Bielza et al. [2].

To put these numbers in perspective, the SRI (Stanford Research Institute) elicitation protocol, still the reference in the field, requires roughly *thirty minutes per parameter* [3], which makes the direct estimation of a model with thousands of entries impractical. The community has responded with a battery of techniques to reduce the parameter count, divorcing parents, sequential refinements, canonical Noisy-OR / Noisy-AND models, and the same authors report that on the jaundice ID itself Noisy-OR achieves a *77 % global reduction* of the probabilistic parameters and up to 97 % on individual distributions [3]. The utility node is in general even harder to elicit: a single utility function like the one used in the neonatal-jaundice ID [12] with five attributes already requires of the order of 5 400 entries and the corresponding pairwise-comparison protocol would demand more than 14 million expert judgements [3]. These figures, far from being anecdotal, are what makes the parameter-elicitation problem the dominant cost of any ID-based DSS deployment.

This thesis addresses precisely this parameter-elicitation problem. We assume that the structure of the ID is given, that all variables are discrete and that the model contains a single value node. Under these assumptions, the question we tackle can be stated as follows:

> *Given the qualitative structure of an Influence Diagram and a small set of decision rules provided by a domain expert, can we automatically recover a parameter vector (CPTs and utilities) that reproduces those rules, thereby relieving the expert from the burden of specifying every numerical entry of the model?*

1.2 Motivation

Several factors make this problem both timely and important.

- **The elicitation bottleneck.** Direct elicitation techniques (reference lotteries, indifference judgements, pairwise comparisons. . .) remain the de-facto standard for the construction of probabilistic decision models [3]. They demand long interviews with scarce experts, are cognitively demanding for the interviewee and are well known to introduce inconsistencies that require subsequent revision. In domains such as clinical medicine, public-health planning or safety-critical engineering, the cost of expert time is the dominant factor preventing the wider adoption of ID-based decision tools.
- **Scarcity of historical data.** A natural alternative to direct elicitation is to learn the parameters of the model from historical data via maximum-likelihood or expectation–maximisation estimators, as originally proposed by Ezawa and Gupta in the context of IDs [10]. However, the regime in which most clinical and engineering problems live is precisely the opposite of what such statistical methods require: there are very few annotated historical cases, but a domain expert is usually able to verbalise a small number of high-level rules of the form "*if condition* $\mathbf{x}$ *holds, then take action* a".
- **Decisions instead of probabilities.** Eliciting rules is cognitively much closer to clinical or managerial practice than eliciting numerical probabilities. Practitioners reason naturally in terms of recommended actions under prototypical scenarios, and reach agreement on rules far more easily than on probability

tables. Exploiting rules as a supervisory signal therefore aligns the elicitation process with the way experts actually think.

- **Use cases.** Concrete domains where the proposed approach is of immediate practical interest include:
  - **Clinical decision support.** Risk stratification, triage, treatment selection or the management of chronic diseases. The neonatal-jaundice ID of [12] is a canonical example, and the demand for evidence-based decision aids in clinical practice is by now well documented [9].
  - **Industrial maintenance and safety.** Inspection scheduling, replacement-vs-repair choices, and safety-critical operations in which expert rules are available but historical incident data are sparse [20].
  - **Public policy and emergency response.** Vaccination campaigns, evacuation plans and resource allocation under uncertainty, where domain rules exist but full probabilistic models do not [16].
  - **Cyber-security and fraud detection.** Operational analysts maintain rule-based playbooks that can be turned into the supervisory signal of an ID-based decision module [8].

1.3 Scope

The field of decision support systems is vast. To keep the contribution focused, the work in this thesis is restricted to:

1. Influence Diagrams with *discrete* chance and decision variables;
2. a *single* value (utility) node on a normalized scale of utilities $[0, 10]$;
3. a *pre-specified* dependency graph (the structure is provided as an input and is not subject to learning);
4. supervision signals consisting of a finite set of *partial decision rules* produced by a domain expert.
5. *non-parametric* characteristics of the probability distributions; for example, $0 < p_i < 1$ for all i, skewness and monotonicity, in ordinal variables (low, medium, high) such as $p_{\text{low}} < p_{\text{med}} < p_{\text{high}}$ or $p_{\text{low}} > p_{\text{med}} > p_{\text{high}}$, mode identification, and other bounds like $a < p_i < b$.

This input to the EDA is, in fact, a future line of research, and it should partially resolve the ill-conditioning of the problem that yields deficient or degenerate solutions.

A wealth of representational extensions has been proposed in the ID literature: asymmetric decision problems (conditional distribution trees, sequential decision diagrams, sequential valuation networks), continuous and hybrid IDs (Gaussian IDs, mixtures of truncated exponentials and polynomials), unconstrained IDs and multi-agent extensions (MAIDs). The review of Bielza, Gómez and Shenoy [4] offers a thorough catalogue of them. Although this thesis deliberately restricts itself to the regular, discrete, single-value setting described above, the extensions are revisited only in the perspectives chapter.

1.4 Deep Learning and Neural Networks

It is impossible to discuss modern artificial intelligence without acknowledging the profound impact of *Deep Learning (DL)* and *Neural Networks*. These architectures currently dominate the landscape of unstructured data processing, achieving unprecedented success in computer vision, natural language processing, and pattern recognition. However, despite their overwhelming popularity, they present severe limitations when applied to the types of decision-support scenarios targeted in this thesis.

- First and most important is the issue of *auditability and interpretability*. Neural networks function inherently as "black boxes"; their knowledge is distributed across millions or billions of meaningless weights. In high-stakes domains such as clinical triage or safety-critical maintenance, a decision-maker cannot blindly trust an automated output. Influence Diagrams, in contrast, are inherently transparent. Every node, arc, and utility value possesses a clear, auditable semantic meaning that experts can inspect, debate, and mathematically validate.
- Secondly, Deep Learning exhibits an extreme *hunger for data*. Training a robust neural network requires vast amounts of labeled historical records to avoid overfitting. As discussed in Section 1.2, the operational reality of many critical domains is characterized by a scarcity of historical data but an availability of expert heuristics. Neural networks are not suited for this data-deprived environment, whereas our proposed approach is explicitly designed to exploit these scarce but high-value expert rules.
- Finally, standard deep models predominantly solve prediction or classification tasks; they do not naturally embed a decision-theoretic framework. An ID explicitly separates the probabilistic beliefs (chance nodes) from the agent's interventions (decision nodes) and their preferences (utility nodes), providing a rigorous mechanism for rational decision-making under uncertainty.

For these reasons, this thesis deliberately steers away from the Deep Learning paradigm. We choose to retain the transparency, rigorous uncertainty management, and use the Influence Diagrams to solve their only major practical drawback: the parameter-elicitation bottleneck.

1.5 Objectives and Contributions

The overall objective of this thesis is to develop and validate a proof-of-concept system that estimates the quantitative parameters of a regular Influence Diagram from a partial expert rule base, using Estimation of Distribution Algorithms as the search engine. To achieve this global goal we set the following specific objectives:

1. Formalise the parameter-recovery problem as a constrained black-box optimisation over the joint space of CPTs and utilities.
2. Design an encoding of the parameter vector compatible with continuous EDAs and capable of guaranteeing the validity of every decoded CPT.
3. Implement the resulting system on top of the EDASPY library [30] and the SMILE inference engine [1].

4. Empirically evaluate the impact of the main hyperparameters (population size, selection ratio, dead iterations, convergence criteria…) as well as different loss functions on the quality of the recovered parameters and on the run-time of the algorithm.
5. Discuss the limits of the proposed approach and outline a set of natural extensions (LLM-informed initialisation, canonical-model parameterisation, parallel EDAs…).

Through a simple model, *"bypass"*, work is done to define the computational estimation methodology for the parameters, and it is tested with a complex real model, *"NHLy"*.

1.6 Thesis Outline

The remainder of the document is organised in five further chapters.

- **Chapter 2** reviews the state of the art: statistical learning from data, direct expert elicitation, evolutionary and generative approaches, and the inverse problem of recovering parameters from rules, positioning the contribution of this thesis.
- **Chapter 3** develops the theoretical framework: Decision Support Systems, the formal definition of an Influence Diagram and its evaluation, the Knowledge-Acquisition problem that motivates this thesis, and a self-contained presentation of Estimation of Distribution Algorithms with the variants relevant for our setting.
- **Chapter 4** describes the implementation of the proof of concept: the architecture of the system, the encoding and decoding of the parameter vector, the design of the fitness function and the configuration of the EDA loop with EDASPY.
- **Chapter 5** reports the experimental study: the benchmark IDs, the protocol of the hyperparameter sweep, the comparison between the rules generated by the optimised model and those provided by the expert, and a discussion of the empirical results.
- **Chapter 6** concludes the work, summarising the contributions and outlining the research directions identified in the state of the art that constitute natural lines of future work.

Chapter 2

State of the Art

The problem of obtaining the numerical parameters of a graphical decision model has been approached from several angles. We organise the literature along four complementary lines and then position the present work within them.

2.1 Statistical learning from data (MLE / EM)

The classical route to parameterising any graphical model is maximum-likelihood estimation. When the data is fully observed the CPTs reduce to relative frequencies; with hidden variables or missing entries the Expectation–Maximisation algorithm is the standard choice. In the specific context of IDs, the work of Ezawa and Gupta [10] combined EM for the CPTs with a greedy search for the causal structure, and extended the procedure to the utility node via regression on observed outcomes.

The approach is already implemented but requires a sizeable corpus of labelled historical cases, which in many real applications is simply not available.

2.2 Direct expert elicitation

When historical data is scarce, the established alternative is the structured elicitation of probabilities and utilities through techniques such as reference lotteries, indifference judgements and canonical models (Noisy-OR, Noisy-AND, additive utilities) [3]. The literature on multi-criteria decision making, including the work of Shahzad on the integration of Bayesian networks and MCDM [28], has produced a comprehensive catalogue of methods to reduce the cognitive load on the expert.

Even with these aids, however, the resulting judgements are known to be systematically biased: the experimental psychology of probability assessment has identified three robust distortions *availability*, *representativeness* and *anchoring* that all degrade the quality of expert-supplied numbers regardless of the analyst's training [3].

The bottleneck therefore persists on two fronts at once: every CPT row and every utility entry must be confirmed by a human, the size of the table grows exponentially with the number of parents, and what is finally written down is itself an unreliable estimate.

2.3 Evolutionary algorithms and metaheuristics

A third line of work treats parameter elicitation as an *inverse optimisation* problem: rather than asking the expert for every parameter, one searches the parameter space for a configuration that reproduces a weak supervisory signal. Genetic Algorithms have been used inside the UPM CIG group for this very purpose [14], and swarm-intelligence techniques (PSO, ACO) have been applied to the related problem of learning unknown behaviours in Interactive Dynamic IDs by Pan et al. [23]. Within evolutionary computation, the family of *Estimation of Distribution Algorithms* (EDAs) [19] has matured into a powerful framework that, instead of recombining individuals via crossover and mutation, explicitly learns a probabilistic model of the promising regions of the search space at every generation.

Recent surveys [18] report consistent improvements over classical GAs in continuous, high-dimensional and constrained problems, and the open-source library EDASPY [30] provides high-quality implementations of several EDAs, including UMDA, EGNA and KEDA.

Evolutionary algorithms, which we use as estimation methods, allow incorporating knowledge from probabilistic models (parametric and non-parametric) and from the specific decision problem domain into the solution, via constraints.

2.4 Generative and LLM-based elicitation

The most recent trend exploits large language models as artificial experts. Prompted with the structure of the network and natural-language descriptions of the variables, models such as GPT-4o, Gemini Pro and Claude 3.5 have been shown to produce CPTs whose KullbackLeibler divergence with respect to expert-elicited tables is comparable to the noise observed across different human experts [21]. Closely related, Pan et al. use Variational Auto-encoders to generate plausible models of the unobserved agents in Interactive Dynamic IDs [22].

These methods are still in their infancy but already suggest powerful hybrid pipelines in which a generative model produces a promising initial configuration that is subsequently refined by a metaheuristic.

2.5 Beyond exact evaluation

The Shachter algorithm computes the optimal policy exactly, but only for regular discrete IDs like the one used in this work. When the diagram grows too large, mixes continuous and discrete variables, or carries constraints that the regular formalism cannot express, exact evaluation is no longer feasible. Two families of methods step in here: *Decision Programming* [26], which rewrites the diagram as an integer optimisation problem and hands it to a standard solver, and *reinforcement learning* [13], which approximates the optimal policy by trial and error on a simulator of the diagram.

Both pursue the same goal given the parameters, find the best decisions which is the mirror image of the problem studied here, where the decisions (the expert rules) are given and the parameters are what we look for. These approaches fall outside the

scope of this thesis, but they are closely related and are worth keeping in mind as the natural counterpart to the parameter-recovery problem we address.

2.6 The inverse problem: rules from CPTs

A complementary line of work, also originated at UPM, approaches the *inverse* of the problem studied in this thesis. Fernández del Pozo, Bielza and Lucas [11] propose the CLUSTER-KBM2L methodology, which starts from a fully specified CPT (more than 8 000 entries in the case of their non-Hodgkin lymphoma ID) and extracts *symbolic prognostic profiles* qualitative rules of the form "high-risk patient with favorable chemotherapy response" by clustering the conditional distributions and synthesizing interpretable descriptions for each cluster.

Both works thus address the same interface between the numerical and the symbolic representation of an ID, but from opposite directions: [11] goes from CPTs to rules, whereas this thesis goes from rules to CPTs.

2.7 Research gap and positioning of this work

The review summarized above leads to a clear conclusion: no published work combines *modern EDAs* with the *reconstruction of ID parameters from a small set of expert decision rules*. The closest precedent is the genetic-algorithm approach of Henríquez [14]. The contribution of the present thesis is to replace the GA with an EDA that *explicitly* models the dependency structure of the parameter space, and to study empirically the impact of this richer probabilistic model on convergence and final accuracy.

Chapter 3

Theoretical Framework

This chapter introduces the theoretical background on which the rest of the thesis relies. We start with Decision Support Systems and their graphical formulation as Influence Diagrams, fix the formal definitions that are used in subsequent chapters and motivate the parameter-estimation problem that we tackle. We then present Estimation of Distribution Algorithms, justify our design choices, including the specific variants and the convergence guarantees that ground the empirical study of Chapter 5.

3.1 Decision Support Systems and Influence Diagrams

A *Decision Support System* (DSS) is a model or piece of software designed to assist decision-makers by facilitating access to relevant data and by applying analytical techniques to poorly structured problems [24, 29]. The role of a DSS is not to replace the decision-maker but to provide structured access to information and a systematic way of evaluating alternatives before a final decision is reached.

Among the many graphical formalisms available, *Influence Diagrams* (IDs) have established themselves as one of the most compact and expressive representations [15, 17]. An ID is a directed acyclic graph (DAG) whose nodes encode the three fundamental ingredients of a sequential decision problem: random variables, controllable choices and preferences, and whose arcs encode conditional dependencies and informational precedence.

3.1.1 Formal definition

Definition1 (Influence Diagram). An *Influence Diagram* is a tuple $\mathcal{D} = (\mathbf{C}, \mathbf{D}, V, \mathcal{E})$ where:

- $\mathbf{C} = \{C_1, \ldots, C_n\}$ is a finite set of *chance nodes*, each modelling a discrete random variable with finite domain Ω_{C_i};
- $\mathbf{D} = \{D_1, \ldots, D_k\}$ is a finite set of *decision nodes*, each with a finite action set $\mathcal{A}_{D_i}$;
- V is a single *utility* (value) node encoding the preferences of the decision-maker;

- $\mathcal{E}$ is a set of directed arcs split into probabilistic, informational and functional arcs.

For the ID to admit a well-defined evaluation procedure the graph must be *regular*. That is, a DAG where the value node has no successors and the decisions can be linearly ordered by a directed path that traverses all of them. Throughout this work all IDs are assumed to be regular.

A concrete example: the bypass influence diagram

We illustrate every notion introduced below on the *bypass* influence diagram, a small clinical model for the management of coronary heart disease that we use throughout the rest of the thesis. It contains six chance nodes (`HeartDisease`, `Angiogram`, `Pain`, `EarlyResults`, `LifeQ`, `Economicalc`), two decision nodes (`HeartSurgery`, `HeartPharma`) and a single utility node. Its qualitative structure is shown in Figure 3.1.

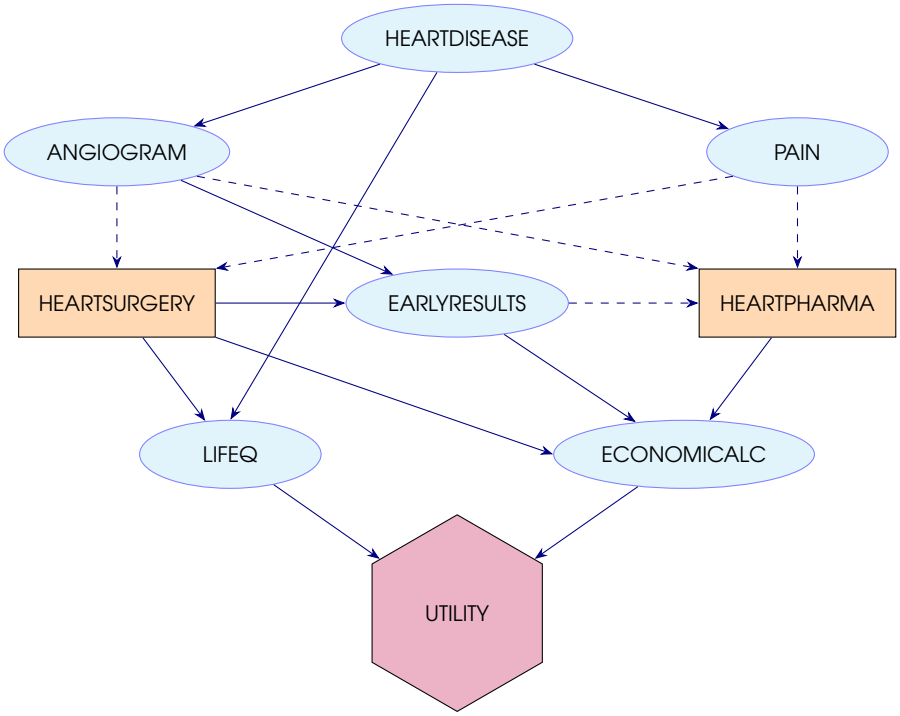

Figure 3.1: Influence diagram of the *bypass* model. Chance nodes are ellipses, decision nodes rectangles and the utility node a hexagon. Solid arcs are probabilistic or functional, dashed arcs are informational.

The Influence Diagram illustrated in Figure 3.1 models a problem with two sequential decisions: first, `Heart Surgery (HS)`, and second, `Heart Pharma (HP)`.

`Angiogram (ANG)` is a diagnostic test and `Pain (PA)` is a symptom; both variables are observed prior to making the decisions. `Early Results (ER)` is conditioned by `ANG` and `HS`, and constitutes known information for the `HP` decision.

The preferences depend on `Life Quality (LQ)` and `Economic Calculation (EC)`. These are conditioned, respectively, by `HS` and `HD` (`Heart Disease`) for `LQ`; and by `ER`, `HS`, and `HP` for `EC`.

The model is regular. Variables LQ and EC have 3 levels, while the rest of the variables are binary.

Regarding the parameterization, there are 9 utility parameters and $2 + 4 + 4 + 8 + 12 + 24 = 56$ parameters modeling the probability distributions. Consequently, there are $56 - 21 = 35$ degrees of freedom. In total, the model is defined by $35 + 9 = 44$ parameters.

3.1.2 Quantitative parameterisation

Each chance node C_i carries a *conditional probability table* (CPT) θ_{C_i} that assigns to every configuration of its probabilistic parents $\mathrm{Pa}(C_i)$ a probability distribution over Ω_{C_i}:

$$\theta_{C_i}\big(\cdot \mid \mathbf{pa}\big) \;\in\; \Delta(\Omega_{C_i}) \qquad \forall \mathbf{pa} \in \prod_{C_j \in \mathrm{Pa}(C_i)} \Omega_{C_j},$$

where $\Delta(\Omega_{C_i})$ denotes the probability simplex over the states of C_i. The utility node carries a real-valued mapping

$$u : \prod_{X \in \mathrm{Pa}(V)} \Omega_X \longrightarrow \mathbb{R},$$

where $\Omega_{D_i} \equiv \mathcal{A}_{D_i}$ for any decision parent. The function u encodes the preferences of the decision-maker in the sense of Multi-Attribute Utility Theory, and when its arguments satisfy the usual independence conditions it admits an additive or multiplicative decomposition that drastically simplifies its elicitation [25]. The collection of all CPTs and the utility table is the *parameter vector* of the diagram, written $\boldsymbol{\Theta}$.

Remark1 (Combinatorial explosion). The number of free entries of a CPT grows multiplicatively with the parents: a node with m parents, each taking r states, and r own states contributes $r^m(r-1)$ free parameters. This exponential growth is the source of the elicitation bottleneck discussed in Chapter 1 and the reason why parameter optimisation rather than exhaustive elicitation is necessary.

A *decision rule* for D_i is a mapping $\delta_i : \prod_{C_j \in \mathrm{Pa}(D_i)} \Omega_{C_j} \longrightarrow \mathcal{A}_{D_i}$ that prescribes an action for every observable context. The collection $\boldsymbol{\delta} = (\delta_1, \ldots, \delta_k)$ over all decisions is called a *policy*. The expected utility of a policy under a parameter vector $\boldsymbol{\Theta}$ is

$$\mathrm{EU}(\boldsymbol{\delta} \mid \boldsymbol{\Theta}) = \sum_{\mathbf{c}} u\big(\mathbf{c}, \boldsymbol{\delta}(\mathbf{c})\big) \prod_{C_i \in \mathbf{C}} \theta_{C_i}\big(c_i \mid \mathbf{pa}(c_i)\big), \tag{3.1}$$

where $\mathbf{c} = (c_1, \ldots, c_n)$ is a complete *scenario*. For each scenario, $\mathbf{pa}(c_i)$ denotes the values its parents take within that same scenario and $\boldsymbol{\delta}(\mathbf{c})$ the actions prescribed by the policy. Thus each scenario is weighted by its joint probability (the product of CPTs) and scored by its utility, and the expectation sums these contributions over every possible state of the space. The *optimal policy* is $\boldsymbol{\delta}^*(\boldsymbol{\Theta}) = \arg\max_{\boldsymbol{\delta}} \mathrm{EU}(\boldsymbol{\delta} \mid \boldsymbol{\Theta})$.

The time complexity of Shachters node-reduction and arc-reversal algorithm is actually exponential in the worst case, but the classical algorithm of Shachter [27] computes $\boldsymbol{\delta}^*(\boldsymbol{\Theta})$ in polynomial time relative to the size of the graph for any fixed $\boldsymbol{\Theta}$ by successively reversing arcs and eliminating nodes from it.

What the parameters and their evaluation look like

The three pieces of the parameterisation become clear when one looks at the tables of the bypass model of Figure 3.1. Table 3.1 shows a real CPT extracted from the model: $P(\texttt{Angiogram} \mid \texttt{HeartDisease})$, which encodes the standard sensitivity/specificity of the angiogram test.

$P(\texttt{Angiogram} \mid \texttt{HeartDisease})$	NEGATIVE	POSITIVE
HeartDisease = ABSENT	0.95	0.05
HeartDisease = PRESENT	0.14	0.86

Table 3.1: Real CPT extracted from the bypass model. Each row is a probability distribution over the two test outcomes.

The utility node `Utility` of the bypass diagram depends on the quality-of-life node `LifeQ` and the cost node `Economicalc`; Table 3.2 shows its full set of values, taken directly from the model. As one would expect, the best outcome (high quality of life, low cost) takes the maximum value, while death together with high cost takes the minimum.

$u(\texttt{LifeQ}, \texttt{Economicalc})$	LOW	MEDIUM	HIGH
LifeQ = DEAD	2.00	0.70	0.05
LifeQ = LIVE2ALQ	3.10	2.90	2.80
LifeQ = LIVE2AHQ	7.80	4.80	2.80

Table 3.2: Utility table of the bypass model.

Given the two tables above (and the rest of the CPTs of the model, not shown), the Shachter algorithm produces a third one: the expected utility of each action of a decision node *conditioned on the information available when that decision is taken*. Table 3.3 shows this output for the `HeartSurgery` decision, whose informational parents are `Pain` and `Angiogram`.

Pain	Angiogram	EU(NO)	EU(YES)
ABSENT	NEGATIVE	**6.98**	3.73
ABSENT	POSITIVE	**5.34**	3.91
PRESENT	NEGATIVE	**6.66**	3.78
PRESENT	POSITIVE	3.43	**4.18**

Table 3.3: Expected utility (EU) of each action of `HeartSurgery` under the parameters of the bypass model, as produced by the Shachter evaluation. The bold value in each row is the optimal action for that information state. Read as a decision rule: *operate only when pain is present and the angiogram is positive; abstain otherwise.*

Schematically, the standard use of an ID flows from CPTs and the utility function towards the optimal policy:

$$(\theta_{C_i},\, u) \xrightarrow{\text{Shachter}} \text{EU} \xrightarrow{\arg\max} \boldsymbol{\delta}^*.$$

The problem this thesis addresses runs the arrows backwards. The expert does *not* supply Tables 3.1 and 3.2; he supplies only a few entries of Table 3.3 as *decision rules* (for example: *do not operate when pain is absent and the angiogram is negative*). The optimizer must recover a pair $(\theta_{C_i},\, u)$ *compatible* with those rules, in the sense that re-running Shachter on the recovered parameters reproduces the expert's choices.

Preference is represented using lotteries for comparisons between uncertain consequences, for example:

$$(\texttt{LQ} = \text{Dead}, \text{EC} = \text{High}; 0.25) \leq (\texttt{LQ} = \text{Live2AHQ}, \texttt{EC} = \text{Low}; 7.5)$$

The very possibility of summarizing the expert's preferences by a single utility function is not gratuitous: it rests on the classical axioms of rational choice.

Definition2 (von Neumann–Morgenstern axioms). A preference relation $\succeq$ over lotteries (probability distributions over outcomes) admits a representation by *expected utility* (i.e. there exists a function u such that $L_1 \succeq L_2 \iff \mathbb{E}_{L_1}[u] \geq \mathbb{E}_{L_2}[u]$) *if and only if* it satisfies the following axioms [31]:

- **Completeness (orderability).** For any two lotteries L_1 and L_2, exactly one of $L_1 \succ L_2$, $L_2 \succ L_1$ or $L_1 \sim L_2$ holds; the agent is never unable to compare two alternatives.
- **Transitivity.** If $L_1 \succeq L_2$ and $L_2 \succeq L_3$, then $L_1 \succeq L_3$.
- **Continuity.** If $L_1 \succeq L_2 \succeq L_3$, there exists a probability $p \in [0, 1]$ such that the agent is indifferent between L_2 and the mixed lottery $p\, L_1 + (1 - p)\, L_3$.
- **Independence (substitutability).** If $L_1 \succeq L_2$, then for any third lottery L_3 and any $p \in (0, 1]$ we have $p\, L_1 + (1 - p)\, L_3 \succeq p\, L_2 + (1 - p)\, L_3$; mixing both options with a common third lottery cannot reverse the preference.

The relevance of the axioms here is conceptual: whenever the recovered pair (θ_{C_i}, u) induces a policy through the maximum-expected-utility rule, that policy is *rational* in the von Neumann–Morgenstern sense by construction. The expert rules, by contrast, carry no such guarantee: they are isolated *if–then* prescriptions that need not derive from any coherent underlying preference order.

This gap raises what we regard as a central theoretical open question behind the whole recovery program:

> *It remains to be seen whether the algorithm can find, for* any *set of expert rules, a parameterization (θ_{C_i}, u) that is simultaneously consistent with those rules and with the axioms of utility theory; or whether some rule sets are intrinsically incoherent, admitting no rational Influence Diagram able to reproduce them.*

3.2 The Knowledge-Acquisition Problem

In practice the structure of the ID is elicited from the expert in a small number of modelling sessions, but the numerical parameterisation $\boldsymbol{\Theta}$ is far harder to obtain. The classical cycle of [2, 3] proceeds in five stages problem identification, enumeration of alternatives, modelling, evaluation and sensitivity analysis and the modelling stage alone accounts for the bulk of the cost. As anticipated in Remark 1, the size of $\boldsymbol{\Theta}$ grows exponentially with the connectivity of the diagram, so an exhaustive elicitation is unrealistic for any diagram of practical size.

We assume instead that the expert can provide a *small set of decision rules*.

Definition3 (Expert rule base). A *rule base* is a finite set

$$\mathcal{R} = \big\{(\mathbf{x}^{(l)}, D_{i_l}, a^{(l)})\big\}_{l=1}^{L},$$

where $\mathbf{x}^{(l)}$ is a (possibly partial) observation vector on the informational parents of decision D_{i_l}, and $a^{(l)} \in \mathcal{A}_{D_{i_l}}$ is the action the expert deems optimal under $\mathbf{x}^{(l)}$. Components marked as "irrelevant" contribute no constraint and the rule must hold for every value they may take.

Given such a rule base we measure the consistency of a parameter vector with the expert by a simple accuracy:

Definition4 (Fitness function). Define the vector of expected utilities of decision D_{i_l} under $\boldsymbol{\Theta}$ and evidence $\mathbf{x}^{(l)}$ as

$$\mathbf{u}^{(l)}(\boldsymbol{\Theta}) = \big(\mathrm{EU}(a \mid \boldsymbol{\Theta}, \mathbf{x}^{(l)})\big)_{a \in \mathcal{A}_{D_{i_l}}}.$$

The generic loss minimised by the EDA is then

$$\mathcal{L}(\boldsymbol{\Theta}) = \sum_{(\mathbf{x}^{(l)}, D_{i_l}, a^{(l)}) \in \mathcal{R}} \ell\Big(\mathbf{u}^{(l)}(\boldsymbol{\Theta}),\ a^{(l)}\Big),$$

where ℓ is a per-rule loss that compares the expected-utility vector of the target decision with the action prescribed by the expert. Different choices of ℓ recover the fitness variants used in this work:

- **Binary** (0–1 loss): $\ell_{\mathrm{binary}}(\mathbf{u}, a) = \mathbf{1}[\arg\max_{a'} u_{a'} \neq a]$.
- **Regret**: $\ell_{\mathrm{regret}}(\mathbf{u}, a) = \max_{a'} (u_{a'} - u_a)$.
- **Margin**: $\ell_{\mathrm{margin}}(\mathbf{u}, a) = \max\big(0,\ \max_{a' \neq a} u_{a'} + m - u_a\big)$.
- **Softmax** (cross-entropy): $\ell_{\mathrm{softmax}}(\mathbf{u}, a) = -\log \mathrm{softmax}(\mathbf{u})_a$.
- **Entropy-regularized**: $\ell_{\mathrm{entropy}}(\mathbf{u}, a) = -\log \mathrm{softmax}(\mathbf{u})_a + \alpha\, H\big(\mathrm{softmax}(\mathbf{u})\big)$.

where m is the margin, α the entropy-regularization weight, and $H(\mathbf{p}) = -\sum_{a'} p_{a'} \log p_{a'}$ the Shannon entropy of the softmax distribution over actions.

The parameter-recovery problem is then the constrained search

$$\boldsymbol{\Theta}^{\star} = \arg\min_{\boldsymbol{\Theta}} \mathcal{L}(\boldsymbol{\Theta}),$$

with $\boldsymbol{\Theta}$ ranging over the product of all valid CPTs and utility tables. The next proposition justifies the use of a derivative-free metaheuristic instead of a gradient-based optimiser.

Proposition1 (Irregularity of the objective). The objective $\mathcal{L}$ of Definition 4 is non-convex with respect to $\boldsymbol{\Theta}$ for every per-rule loss ℓ considered in this work. For the 0–1 (BINARY) loss it is, in addition, piecewise constant and discontinuous; for the REGRET and MARGIN losses it is continuous but non-differentiable on the set where the optimal action of a decision switches; for SOFTMAX and ENTROPY it is piecewise smooth.

Sketch. ***Non-convexity.*** For a chain of two chance nodes the expected utility contains a term proportional to the product of two parameters, $p\,q$. The map $(p, q) \mapsto p\,q$ is not convex: at $(1, 0)$ and $(0, 1)$ it equals 0, yet at the midpoint $(\frac{1}{2}, \frac{1}{2})$ it equals $\frac{1}{4} > 0$, violating the convexity inequality. Since $\mathbf{u}^{(l)}(\boldsymbol{\Theta})$ generically involves such products (a node multiplies the probabilities of its parents) and a normalisation by the probability of the evidence, $\mathcal{L}$ is non-convex.

Regularity. Within any region of $\boldsymbol{\Theta}$ where the optimal action of every decision is fixed, $\mathbf{u}^{(l)}$ is a rational function of the parameters and is therefore smooth. At the boundaries between regions the maximising action switches: the value of the optimal policy is non-differentiable there. Composing with the per-rule loss, the 0–1 loss jumps discontinuously at those boundaries, the REGRET and MARGIN losses stay continuous but non-differentiable, and the SOFTMAX and ENTROPY losses are smooth away from them. □

A gradient of $\mathcal{L}$ thus exists almost everywhere (except for the discontinuous BINARY loss) and could in principle be obtained by finite differences. We nevertheless adopt a population-based, derivative-free strategy for three reasons:

- The landscape is strongly non-convex and highly degenerate (many parameter vectors reproduce the same rules) so local gradient information does not lead to the global optimum.
- The evaluation engine returns only utility values, so each numerical gradient would cost $O(d)$ additional evaluations in a space of dimension $d = |\boldsymbol{\Theta}|$.
- And for the BINARY loss the gradient is identically zero almost everywhere.

3.3 Estimation of Distribution Algorithms

Estimation of Distribution Algorithms (EDAs) [18, 19] are a class of evolutionary metaheuristics in which the classical crossover-and-mutation operators are replaced by the *construction and sampling of an explicit probabilistic model* of the search space. At every generation a probability distribution is fitted to the best individuals of the current population and used to generate the next generation. Algorithm 1 states the generic scheme.

Algorithm1 (Generic EDA).

1: Initialise a population $\mathcal{P}_0$ of N individuals from a prior on the search space.
2: $t \leftarrow 0$
3: **repeat**
4: Evaluate f on every $\boldsymbol{\Theta} \in \mathcal{P}_t$.
5: $\mathcal{P}_t^{\text{sel}} \leftarrow$ top-S individuals of $\mathcal{P}_t$.
6: Estimate a probabilistic model $\hat{p}_t$ based on $\mathcal{P}_t^{\text{sel}}$.
7: Sample N new individuals from $\hat{p}_t$ to form $\mathcal{P}_{t+1}$.
8: $t \leftarrow t + 1$
9: **until** stopping criterion is met
10: **return** best individual found.

In words, the algorithm keeps a *population* of candidate solutions and repeatedly (i) scores each one with the fitness f, (ii) keeps the top-S elite, (iii) fits a probabilistic

model $\hat{p}_t$ to that elite, and (iv) samples the next generation from it. It thus replaces the crossover and mutation of a genetic algorithm by *learning and sampling a distribution* over the good solutions, which is what lets it exploit statistical regularities of the search space.

A candidate solution (an *individual*) is a real-valued vector $\boldsymbol{\phi} = (\phi_1, \ldots, \phi_d) \in \mathbb{R}^d$ whose components stack, in a fixed order, the raw value of every free CPT entry and every utility entry of the diagram. The search itself is *unconstrained* each ϕ_j lives in the whole real line $\mathbb{R}$, and the bounds $[-5, 5]/[-10, 10]$ constrain only the initial sampling and it is a decoder (Chapter 4) that turns $\boldsymbol{\phi}$ into a valid parameter vector $\boldsymbol{\Theta}$, enforcing the two constraints of the model: every CPT row must be a probability distribution (non-negative, summing to one) and every utility must lie inside the expert range $[u_{\min}, u_{\max}]$. As an example, for a diagram with a single binary chance node conditioned on a binary parent (2 rows of 2 entries) and 2 utility values, one valid individual is

$$\boldsymbol{\phi} = \big(\underbrace{1.2,\ -0.4}_{\text{CPT row 1}},\ \underbrace{0.1,\ 0.3}_{\text{CPT row 2}},\ \underbrace{2.0,\ -1.5}_{\text{utilities}}\big),$$

which the decoder maps (row-wise softmax and a rescaled sigmoid) to, e.g., CPT rows $(0.83, 0.17)$ and $(0.45, 0.55)$ and utilities $(8.8,\ 1.8)$ on the range $[0, 10]$ a fully valid model regardless of the raw values sampled.

The defining choice of an EDA is the family used for $\hat{p}_t$: richer families capture more structure of the search space at the expense of a higher estimation cost.

3.3.1 Dependency Models in EDAs

Because the parameter vector of an ID lives in a continuous space (every CPT entry and every utility value is a real number), only continuous EDAs are relevant. The four models used in this work, all available through EDASPY [30], span the spectrum from a fully factorised parametric model to a full-covariance multivariate Gaussian, with a structured intermediate and a non-parametric alternative on the univariate side. We denote an individual by $\boldsymbol{\phi} = (\phi_1, \ldots, \phi_d)$ and the size of the selected elite by $S = |\mathcal{P}_t^{\text{sel}}|$.

UMDA$_\mathbf{c}$ — Univariate Marginal Distribution Algorithm. Each dimension is modeled by an independent Gaussian [19]:

$$\hat{p}_t(\boldsymbol{\phi}) = \prod_{j=1}^{d} \mathcal{N}\big(\phi_j;\, \hat{\mu}_t^j,\, (\hat{\sigma}_t^j)^2\big),$$

whose mean and variance are the sample moments of the selected individuals,

$$\hat{\mu}_t^j = \frac{1}{S} \sum_{\boldsymbol{\phi} \in \mathcal{P}_t^{\text{sel}}} \phi_j, \qquad (\hat{\sigma}_t^j)^2 = \frac{1}{S} \sum_{\boldsymbol{\phi} \in \mathcal{P}_t^{\text{sel}}} (\phi_j - \hat{\mu}_t^j)^2.$$

The model is the cheapest of the family ($O(d \cdot S)$ per generation, both for fitting and for sampling) but its independence assumption means it cannot capture any dependency between the parameters of the diagram: the search effectively assumes that good values for one CPT entry give no information about good values for another.

EMNA — Estimation of Multivariate Normal Algorithm. The opposite end of the parametric spectrum: the whole population is modelled by a single multivariate Gaussian [19],

$$\hat{p}_t(\boldsymbol{\phi}) = \mathcal{N}\big(\boldsymbol{\phi};\, \hat{\boldsymbol{\mu}}_t,\, \hat{\boldsymbol{\Sigma}}_t\big),$$

with sample mean and *full covariance*,

$$\hat{\boldsymbol{\mu}}_t = \frac{1}{S}\sum_{\boldsymbol{\phi}\in\mathcal{P}_t^{\text{sel}}} \boldsymbol{\phi}, \qquad \hat{\boldsymbol{\Sigma}}_t = \frac{1}{S}\sum_{\boldsymbol{\phi}\in\mathcal{P}_t^{\text{sel}}} (\boldsymbol{\phi}-\hat{\boldsymbol{\mu}}_t)(\boldsymbol{\phi}-\hat{\boldsymbol{\mu}}_t)^\top.$$

EMNA captures every pairwise linear dependency between parameters and is therefore strictly more expressive than UMDA. The price is double: the cost grows to $O(d^2 \cdot S)$ for the covariance estimate, plus a $O(d^3)$ Cholesky factorisation of $\hat{\Sigma}_t$ used to sample (and the truncated population must be large enough, $S > d$ to keep $\hat{\boldsymbol{\Sigma}}_t$ non-singular) [19].

EGNA — Estimation of Gaussian Networks Algorithm. A middle ground between UMDA and EMNA: a multivariate Gaussian whose factorisation is governed by a Bayesian network $\mathcal{G}$ *learnt from the elite at every generation* [19]. Each variable conditions only on its parents in $\mathcal{G}$,

$$\hat{p}_t(\boldsymbol{\phi}) = \prod_{j=1}^{d} \mathcal{N}\Big(\phi_j;\; \beta_{j,0} + \sum_{k\,\in\,\mathrm{pa}_{\mathcal{G}}(j)} \beta_{j,k}\,\phi_k,\; \sigma_j^2\Big),$$

i.e. a linear regression of ϕ_j on its graph parents with Gaussian residuals. The structure $\mathcal{G}$ is chosen with a score (typically BIC) and a greedy search over DAGs [19]. EGNA recovers UMDA when $\mathcal{G}$ has no edges and approaches EMNA when $\mathcal{G}$ becomes a complete DAG; in practice the learnt graph is sparse, which keeps the cost reduced and makes the model robust if S is not large enough for EMNA to estimate a full covariance reliably.

KEDA (Univariate) — Kernel-density Estimation of Distribution Algorithm. A non-parametric alternative on the univariate side: each marginal is fitted with a kernel density estimator from the elite [30],

$$\hat{p}_t(\boldsymbol{\phi}) = \prod_{j=1}^{d} \hat{p}_t^{\,j}(\phi_j), \qquad \hat{p}_t^{\,j}(\phi_j) = \frac{1}{S\,h_j}\sum_{\boldsymbol{\phi}^{(s)}\in\mathcal{P}_t^{\text{sel}}} K\Big(\frac{\phi_j-\phi_j^{(s)}}{h_j}\Big),$$

where K is a kernel (typically Gaussian) and h_j is a bandwidth chosen, for instance, by Silverman's rule. Like UMDA it ignores cross-parameter dependencies, but it drops the Gaussian assumption per dimension and can therefore fit skewed or multimodal marginals (a setting in which a single Gaussian would smear over distinct modes). Its cost is again $O(d \cdot S)$, with the bandwidth as an additional implicit hyperparameter that trades smoothness against detail.

Together the four variants span the spectrum from *fully factored* (UMDA$_c$, KEDA) to *fully joint* (EMNA), with EGNA as a structured intermediate, and from *parametric* (UMDA$_c$, EMNA, EGNA) to *non-parametric* (KEDA) on the per-dimension side. Which combination suits the parameter-recovery problem of this work is far from obvious a priori.

3.3.2 Convergence of the Algorithm

The reason an EDA makes progress is intuitive and needs no heavy machinery. At each generation it keeps only the best fraction of the population a *truncation* selection that retains the top $\mu \in (0, 1)$ of the individuals and refits the probabilistic model $\hat{p}_t$ to that elite. Because the selected individuals concentrate in the better-scoring regions of the search space, the model fitted to them places its mass there, so the next population, sampled from $\hat{p}_t$, is on average better than the previous one.

Repeating the loop has two coupled effects. The model *drifts* towards regions of lower cost, generation after generation, while its variance *contracts* as the surviving individuals become more alike. The drift is what improves the solutions; the contraction is what makes the search eventually settle on a concrete configuration. When the variance has shrunk to the point where new samples barely differ from one another, the population has converged.

The selection ratio μ balances these two effects. A small μ keeps only a handful of top individuals: the model contracts quickly and the search converges fast, but it may lock onto the first good region it finds. A large μ keeps almost the whole population, so little information is extracted from the cost function and the search behaves almost like random sampling. A useful value lies in between and is tuned empirically in Chapter 5.

This argument is deliberately informal. It explains why the distribution concentrates on good regions, not that the region found is the global optimum on the non-convex, degenerate landscape of Proposition 1 it generally need not be. What matters in practice is that the EDA reliably drives the cost down towards high-quality solutions, which is what the experiments of Chapter 5 confirm. The only additional requirement specific to our problem is that every sampled individual be turned into a valid set of CPTs each row a probability distribution which is handled at the decoding stage described in Chapter 4.

The solution reached must be validated by the expert and consider several iterations, revising/expanding the input rule base, adding constraints to the parameter space, and possibly revising the dependency graph.

Chapter 4

Estimation of the influence diagram parameters

This chapter describes the implementation used to estimate the parameter vector $\boldsymbol{\Theta}$ of a regular Influence Diagram from a partial expert rule base $\mathcal{R}$.

4.1 Parameter tuning system design

The system is built on top of two external components: the SMILE reasoning engine and its `pysmile` Python library, which provide the `.xdsl` parser and the Shachter evaluation algorithm [1, 27]; and EDAspy, the open-source library of Soloviev et al. [30] that implements the EDA variants discussed in Chapter 3. Around these dependencies the system is organised in four functional layers:

1. **Model loading and structural extraction.** The `.xdsl` file is parsed into an internal node dictionary that holds the graph structure and a `NumPy` reference to every numerical table.
2. **Rule-base parsing.** The expert rule base is read from a CSV file and translated into a list of (**evidence**, D_{i_l}, $a^{(l)}$) triples.
3. **Fitness evaluation.** For each candidate parameter vector, the Shachter engine is invoked once per rule and the losses are aggregated into the score the EDA tries to minimise.
4. **EDA-based optimisation.** The continuous EDA variants of EDAspy drive the search over an encoded parameter vector ϕ.

4.2 Model structural extraction and rule parsing

The diagram is read once with the `pysmile` library, which exposes every node, its parents and its CPT. The pipeline keeps a reference to each table so that a candidate solution can be written back in place from the parameter vector and the diagram re-evaluated without rebuilding the network every time; the evaluation is by far the most expensive operation, so it is repeated as little as possible.

The expert rules are synthetically created by evaluating the network multiple times, each with a different possible combination of events. Then they are stored in a CSV file with one rule per row. Each row records the observed state of the relevant variables and the action the expert considers optimal for the target decision; a sign convention distinguishes the observed variables from the target decision and marks the columns that play no role in that rule (any node that is not a parent of the decision node). Only rows that actually name a target decision are kept.

The result is a list of triples:

$$(\text{evidence},\ \text{decision},\ \text{expected action})$$

that the scoring step will check one by one.

4.2.1 A real rule from the bypass model

Throughout the rest of the chapter we ground the description on the *bypass* influence diagram already introduced in Chapter 3 (Figure 3.1, Tables 3.1–3.3). Here we focus on the operational side that Chapter 3 does not discuss: how a single line of the expert rule base is turned into a constraint the optimiser can check.

The rule base is a plain text database in CSV format with one row per rule and one column per node of the diagram. Sign convention: a positive integer $+v$ encodes that the corresponding chance node is observed in its $(v-1)$-th state; a negative integer $-v$ identifies the target decision node and the action $(|v|-1)$ the expert prescribes for it; the value 0 marks the node as irrelevant for that particular rule (in practice, it is a node that is not an informational parent of the target decision). For the bypass model the eight columns are:

```
ANGIOGRAM, EARLYRESULTS, ECONOMICALC, HEARTDISEASE,
      LIFEQ, PAIN, HEARTPHARMA, HEARTSURGERY,
```

and the very first row of `reglas_generadas.csv` reads

```
1, 1, 0, 0, 0, 1, -1, 0
```

which, decoded with the state mappings of the model, becomes the explicit rule

> *If* `Angiogram` = NEGATIVE *and* `EarlyResults` = COREM *and* `Pain` = ABSENT, *then* `HeartPharma` = NO.

Evaluating this rule means: write the decoded Θ into the model, enter the three observed states as evidence, run Shachter on `HeartPharma` and check that its optimal action is NO. The fitness of the candidate is the aggregated per-rule loss over the whole `reglas_generadas.csv`, computed in the next section.

4.3 Encoding and decoding the parameters

The EDA searches over plain real vectors, but the parameters of an ID are constrained: every CPT row must be a probability distribution and the utilities must stay on a bounded scale. The encoding keeps the two worlds apart. The optimiser works on a *raw* vector ϕ that lives in the whole real space: its entries are only initialised uniformly within symmetric bounds in our runs $[-5, 5]$ for the CPT entries

and $[-10, 10]$ for the utilities and are then sampled freely over $\mathbb{R}$, while a *decoder* turns that raw vector into a valid model. As mentioned earlier, a fixed node ordering encoded in a dictionary tells the decoder which slice of ϕ belongs to each table.

Decoding is where the constraints are enforced, through two squashing functions applied row by row:

- **Chance nodes — softmax with ε-floor.** Each CPT row is first softened by the softmax, $\tilde{\theta}_j = \mathrm{softmax}(\phi_{\text{row}}/T_c)_j$, which already returns non-negative values that sum to one. To prevent any entry from being exactly 0 or 1 (degenerate CPTs that make subsequent evaluations brittle and forbid recovery from a wrong commitment), we additionally floor each entry at a small constant ε and renormalise the row:
$$\hat{\theta}_j = \frac{\max(\varepsilon,\, \tilde{\theta}_j)}{\sum_k \max(\varepsilon,\, \tilde{\theta}_k)}.$$
With $\varepsilon = 10^{-3}$ this guarantees $\hat{\theta}_j \in [\varepsilon',\, 1 - \varepsilon']$ for some small $\varepsilon' > 0$ depending on ε and the row cardinality, while preserving the simplex constraint.
- **Utility node — sigmoid.** Each free utility entry is the sigmoid of its raw value, rescaled to the chosen range $[u_{\min}, u_{\max}]$ (here $[0, 10]$): $\hat{u} = \sigma(\phi/T_u)\,(u_{\max} - u_{\min}) + u_{\min}$. Two entries are then pinned to the best and worst outcomes declared by the expert. Because the sigmoid is asymptotic — it maps any finite raw value into the *open* interval $(0, 1)$ — the decoded utilities stay strictly inside the interior of the $[u_{\min}, u_{\max}]$ range, i.e. they never reach exactly $u_{\min}$ nor $u_{\max}$ except at the two pinned entries.

Proposition2 (Validity of the decoded model). For every raw vector ϕ the decoded tables form a valid parameterisation: softmax guarantees that each CPT row is non-negative and sums to one, and the rescaled sigmoid keeps every utility inside $[u_{\min}, u_{\max}]$. The Shachter engine can therefore be evaluated unconditionally, with no need to reject or repair samples.

However, these two squashing functions are not innocuous; their consequences are worth explaining in detail:

- **No repair step.** Because softmax and sigmoid land on the feasible set by construction, the EDA can explore an *unconstrained* real space while the decoded model will remain always valid (there will never be a sample to clip or discard). This is what makes the row-wise validity of Proposition 2 come for free.
- **Temperatures control sharpness.** The temperatures T_c, T_u rescale the raw values before squashing. A low T_c pushes a CPT row towards a near-deterministic distribution (all the mass on one state); a high T_c flattens it towards the uniform. The sigmoid behaves analogously for the utilities, sharpening or softening the contrast between outcomes. In summary, the temperature controls the entropy of the parameter vector and, as we will see later, introduces a trade-off effect for certain fitness functions.
- **Monotonicity and saturation of the utility.** The sigmoid is monotone, so the *ordering* of the raw utility values is preserved in the decoded utilities (which is all that matters, since the optimal policy depends only on that ordering). Its saturation, however, means that most of the sensitivity is concentrated near the

midpoint of the range: extreme raw values barely move the decoded utility, so the optimiser has fine control over close calls and coarse control at the extremes.

Finally, the two pinned utility entries are not cosmetic. Because the optimal policy depends only on the ordering of the utilities, u is identifiable only up to a positive affine transformation; fixing the best and worst outcomes removes that ambiguity and stops the EDA from drifting along a direction that does not change the policy.

4.4 Scoring a candidate

A candidate is scored by how well the policy it induces reproduces the expert rules. This is the mean per-rule loss $\mathcal{L}(\boldsymbol{\Theta})$ of Definition 4, which EDASPY minimises directly. A central point is that this loss is *not unique*: the per-rule term ℓ is a configurable choice, and the implementation exposes the whole family introduced in Chapter 3 the 0–1 BINARY loss, REGRET, MARGIN, SOFTMAX and ENTROPY whose effect on convergence and final accuracy is compared experimentally in Chapter 5. The simplest member is the 0–1 loss, whose mean is just the fraction of rules the model gets wrong:

Definition5 (Rule-accuracy loss). With the 0–1 per-rule loss, the objective reduces to

$$\mathcal{L}\big(\boldsymbol{\Theta}(\boldsymbol{\phi})\big) = \frac{1}{|\mathcal{R}|} \sum_{(\mathbf{x}^{(l)}, D_{i_l}, a^{(l)}) \in \mathcal{R}} \mathbf{1}\Big[\delta^*_{i_l}\Big(\boldsymbol{\Theta}(\boldsymbol{\phi}), \mathbf{x}^{(l)}\Big) \neq a^{(l)}\Big],$$

where $\boldsymbol{\Theta}(\boldsymbol{\phi})$ is the decoded vector and $\delta^*_{i_l}$ is the action the Shachter engine returns under the evidence $\mathbf{x}^{(l)}$. The smoother variants replace the indicator by the corresponding ℓ of Chapter 3, exposing more gradient-like information to the search.

Whatever the loss, evaluating one candidate means decoding it into the tables and, for each rule, entering its evidence, running the diagram and inspecting the recommended action.

4.5 The EDA based optimizer

EDASPY [30] is the reference open-source EDA library of the UPM Computational Intelligence Group (CIG)[1]; it implements the univariate and multivariate variants reviewed in Chapter 3 behind a common interface. The pipeline plugs into this library and exposes four of its continuous variants:

- `UMDAc` independent Gaussian marginals (no cross-parameter dependencies).
- `EMNA` multivariate Gaussian with full covariance (all pairwise dependencies, no structure).
- `EGNA` Gaussian Bayesian network with structure learnt every generation (a sparse multivariate model).
- `KEDA (univariate)` kernel-density marginals (no Gaussian assumption per dimension).

[1] `https://cig.fi.upm.es/`

Together they form a spectrum from fully factorized to fully multivariate, and from parametric to non-parametric. Which one is preferable for the parameter-recovery problem is not obvious a priori (full covariance is more expressive but more costly to estimate). We therefore leave the choice to the empirical comparison of Chapter 5. Richer variants such as SPEDA, also implemented in EDASPY, are left to the perspectives chapter.

The run is governed by the few hyperparameters of Table 4.1. The search stops early when the best cost does not improve for T_{dead} consecutive generations, which avoids wasting time once the population has converged.

Hyperparameter	**Symbol**	**Role**
`optimizer_type`	—	UMDA, EMNA, EGNA or KEDA
`size_gen`	N	Individuals per generation
`max_iter`	T	Maximum number of generations
`dead_iter`	T_{dead}	Early-stopping patience
`alpha`	μ	Truncation selection ratio
`elite_factor`	μ_{el}	Fraction of best individuals preserved

Table 4.1: Main hyperparameters used in this work.

The first population is drawn *uniformly* within the symmetric raw bounds. This choice is deliberate. Because the bounds are centered at zero and both squashing functions are symmetric, a raw value of zero decodes to a uniform CPT row and to the mid-range utility. The search therefore starts *unbiased*, with no built-in preference for any state of a chance node or any action of a decision. Uniform sampling also scatters the initial individuals across the whole raw space rather than around a single guess, so the EDA explores broadly before its model contracts which matters on the multimodal, degenerate landscape of this problem, where many parameter vectors reproduce the same rules and an early commitment to one region would be premature.

A natural extension, supported by EDASPY and discussed in the perspectives chapter, replaces this uninformed start: an LLM-generated warm start [21].

Chapter 5

Experimental Study

This chapter evaluates the pipeline of Chapter 4 on two real influence diagrams of clinical origin. The *bypass* diagram, already introduced in Section 4.2.1 as the running example of the implementation, plays the role of a small sanity-check benchmark. The *NHLy* diagram, presented in the next section, is one order of magnitude larger and constitutes the main benchmark of the empirical study.

5.1 Benchmark networks

The experimental study uses two influence diagrams of clinical origin and very different size: the small *bypass* diagram, which serves as a controlled benchmark, and the much larger *NHLy* diagram, which stresses the method on a realistic parameter vector. This section introduces both.

5.1.1 Bypass: management of coronary heart disease

The *bypass* diagram was already presented in Section 4.2.1 as the illustrative model used to explain the implementation. It is reused here as a small, controlled benchmark on which the parameter vector is small enough for a finer study of the encoding, the decoding, the loss and the EDA variant.

5.1.2 NHLy: management of non-Hodgkin lymphoma

The *NHLy* influence diagram models the clinical management of gastric non-Hodgkin lymphoma [5]. Its qualitative structure is shown in Figure 5.1. It comprises sixteen chance nodes covering disease, treatment-side-effect and outcome variables, three decision nodes (`Surgery`, `Helicobacter treatment` and `CT-RT schedule`) and a single utility node. Because of its size and connectivity, the parameter vector Θ associated with the diagram is of the order of several thousand entries (Table 5.1), which makes the elicitation bottleneck discussed in Chapter 1 fully tangible and the EDA-based recovery the only practical route to populating the network.

	CPT entries		Utility ent.	Total $\lvert\Theta\rvert$	
	all	free		all	free
bypass	86	57	9	95	66
NHLy	8 282	5 131	144	8 426	5 275

Table 5.1: Size of the parameter vector for the two benchmark networks. "All" counts every entry of every CPT row; "free" discounts the one-per-row redundancy introduced by the probabilities constraint $\sum_j \theta_j = 1$.

To make these parameters concrete, Tables 5.2 and 5.3 show, respectively, one real CPT and a slice of the utility table of *NHLy*; the nodes they involve are marked with an asterisk in the diagram of Figure 5.1. Table 5.2 gives $P($`Perforation` $\mid$ `CT-RT schedule`$)$: the risk of perforation grows monotonically with the aggressiveness of the schedule. Table 5.3 shows how the utility depends on the five-year outcome and the patient's general health status, with the remaining three parents fixed: survival dominates the value and the health status only modulates it.

$P($`Perforation` $\mid$ `CT-RT schedule`$)$	NO	YES
`CT-RT` = NONE	0.99	0.01
`CT-RT` = RADIOTHERAPY	0.97	0.03
`CT-RT` = CHEMOTHERAPY	0.95	0.05
`CT-RT` = CHEMO $\rightarrow$ RADIO	0.92	0.08

Table 5.2: A real CPT of *NHLy*: probability of `Perforation` conditioned on the `CT-RT schedule`. Each row is a probability distribution over the two outcomes. The two nodes involved are starred (*) in Figure 5.1.

$u($`FiveYearResult`, `GeneralHealthStatus`$)$ (parents fixed: `HelicobacterTreatment`=NO, `Surgery`=NONE, `CT-RT`=NONE)			
	POOR	AVERAGE	GOOD
`FiveYearResult` = ALIVE	0.90	0.95	1.00
`FiveYearResult` = DEATH	0.15	0.18	0.25

Table 5.3: A slice of the *NHLy* utility table (which has 144 entries over five parents). Three parents are held fixed and the value is shown as a function of the five-year result and the general health status: survival dominates the utility and the health status only modulates it. The nodes involved are starred (*) in Figure 5.1.

The Influence Diagram illustrated in Figure 5.1 models a complex medical treatment process, involving three sequential decisions: `Helicobacter Treatment`, `Surgery`, and `CT RT Schedule` (chemotherapy and radiotherapy).

Unlike simpler diagnostic models, this diagram captures a broad spectrum of clinical chance variables. These encompass the patient's baseline profile (such as `Age`, `General Health Status`, and `Clinical Stage`), intermediate physiological responses and severe complications (like `Hemorrhage`, `BM Depression`, or `Perforation`), and various survival metrics across different treatment stages.

Ultimately, the overall clinical preference is captured by a single *Utility* node that evaluates the consequences of the entire chosen clinical pathway, culminating in long-term assessments such as the `Five Year Result`.

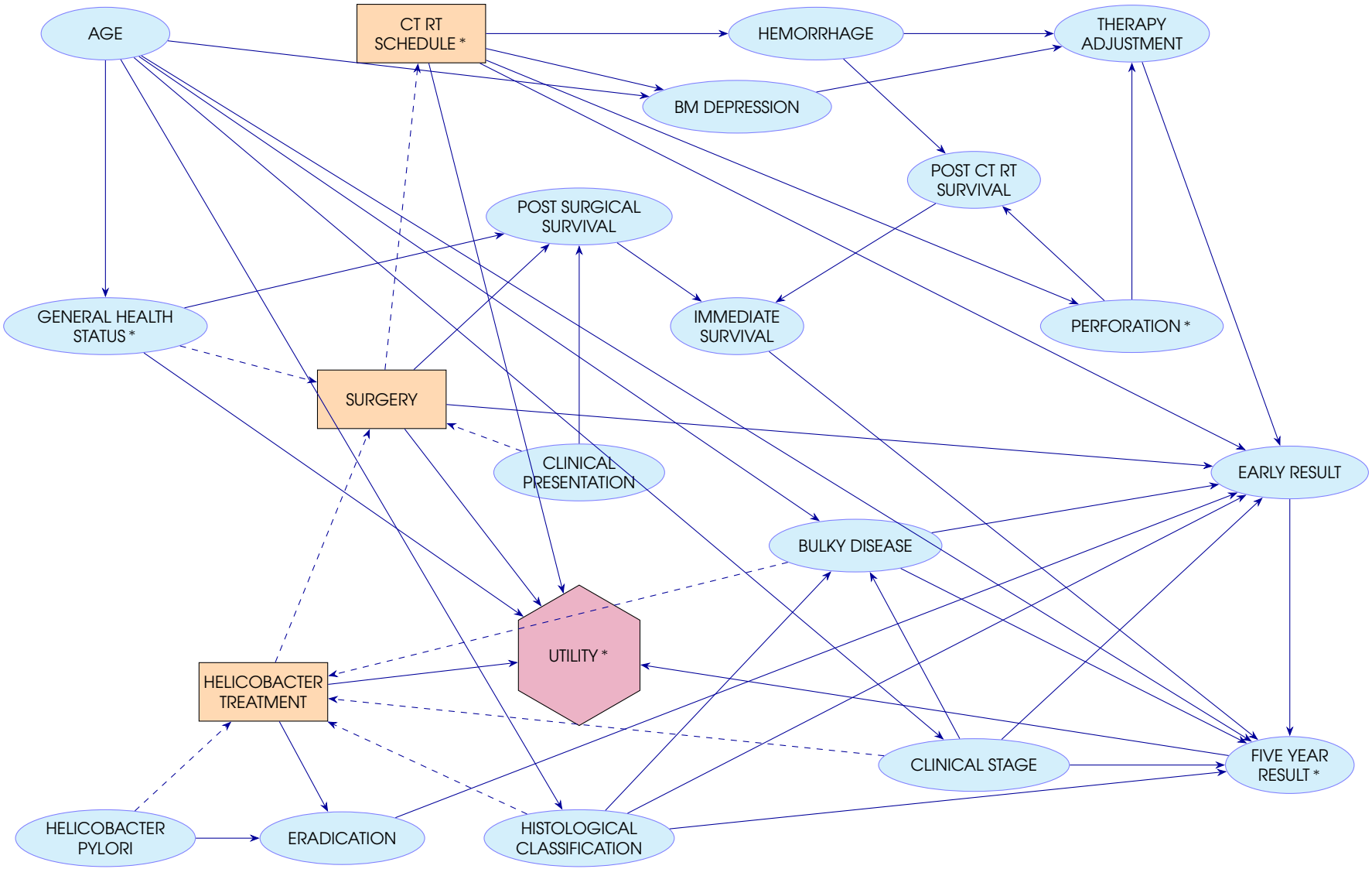

Figure 5.1: Influence diagram of the *NHLy* model for the management of gastric non-Hodgkin lymphoma. Nodes marked with an asterisk (*) are those involved in the example CPT and utility slice of Tables 5.2 and 5.3.

5.2 Experimental protocol

The goal of the recovery is operational: to reproduce the *decisions* the expert would take. Everything in this chapter is measured towards that goal. As mentioned previously, the rule bases are *synthetic* (generated by evaluating the original diagram, as described in Chapter 4) and then, the ground-truth model is known, so the recovery can be judged quantitatively.

Here, several important qualifications must be made. Not all recovered models are equally informative: rule bases do not fully determine every component of an influence diagram (solution space is highly degenerated), and different recovery methods vary in their ability to reconstruct the underlying model depending on the initialization and the given rules. Consequently, this dictates the hierarchy of the metrics below.

5.2.1 Experiment metrics

Each run is summarised by a small set of quantities, ordered here by how much they matter for the recovery problem. They are computed on the *satisfiers* of a run — the individuals that reproduce the training rules — by taking their mean, and aggregated over the independent repetitions with different rules and random initialization.

1. **Rule accuracy (the primary measure).** The fraction of the *whole* rule base that the recovered policy reproduces (equivalently $1 - \mathcal{L}$ for the 0–1 loss of Definition 5). Reproducing the expert's decisions is the actual objective, and in a real elicitation it is the only signal available. We report its **mean** over the repetitions.

2. **Dispersion of the accuracy (reliability).** EDAs are stochastic methods; therefore, their quality depends not only on their performance but also on their repeatability and robustness to variations in the rule base and parameter initialization: one that swings between 80% and 100% across seeds is worse than one that sits steadily at 95%. We therefore report the **standard deviation** of the accuracy alongside its mean.

3. **Recovery time (cost).** How expensive it is to reach that accuracy: the number of generations to convergence and the CPU time, measured as process CPU seconds rather than wall-clock, so that optimizers (EDAs) with very different per-generation cost are compared fairly. This metric becomes particularly important for large models, where recovery time can increase dramatically with the number of parameters.

4. **Probability reconstruction (a secondary objective).** The mean squared error of the recovered conditional probability tables against the ground truth, $\mathrm{MSE}_{\mathrm{CPT}}$. Unlike the rules, this rewards recovering the *actual* probabilities, which are objective quantities with a true target; doing so is a stronger, desirable outcome, but it is subordinate to reproducing the rules and only meaningful where the chance model is itself optimized (in most experiments with NHLy, only utilities will be recovered, leaving appart probabilities, to reduce the computational cost).

The ordering is deliberate. Quality is judged first by *how many rules* are reproduced and *how reliably*, then by *how cheaply*; recovering the probability tables is a welcome bonus.

5.2.2 Several repetitions

An EDA recovery is stochastic in two distinct ways, so a single run is a sample of a random variable rather than a fixed measurement:

- **The optimizer is itself randomized:** it starts from a random initial model (individuals drawn from a uniform distribution) and at each generation re-estimates the model from the highest-accuracy subset of the population before sampling the next one, so even on the very same rules two runs may follow different trajectories and may settle on different policies (as Section 5.3 examines in detail).
- **Which rules are exposed is itself a random draw:** each run trains on a given fraction of the rule base, and the particular subset of decision rules shown changes what can be identified, since different subsets constrain the policy to different degrees.

Reporting a single run would therefore conflate the quality of a method with the luck of one seed. To turn the metrics of Section 5.2.1 into meaningful estimates, we run every configuration several independent times, each with a different random seed that simultaneously redraws the initialization and the rule sample: the **mean** over repetitions estimates the expected accuracy (the primary metric) and their **standard deviation** estimates the reliability (the second metric), neither of which is computable from a single run.

It is worth noting that, in a real-world application, it would be the expert who decides which rules to prioritize, according to their importance and how well they capture the most critical decisions.

5.2.3 Loss functions under study

The recovery minimizes the aggregated per-rule loss of Definition 4; the experiments in this chapter sweep its five instances, which differ only in *how* they penalize for each rule the candidate gets wrong. They are not redundant, they shape the different strategies, and the purpose of comparing them is to find out whether a smoother, more informative signal helps the EDA to converge. Briefly:

- **`binary`** (the 0–1 loss). Counts how many training rules the candidate fails to reproduce, scoring nothing for *how* wrong each one is. It is the strictest criterion and the one that most faithfully reflects the actual objective (rule accuracy), but it yields a loss landscape with many flat regions, which can make it more difficult to converge to local optima. We test it as the natural baseline and the one all other losses must beat to be worth their extra complexity.
- **`regret`** and **`margin`**. Instead of a yes/no verdict they measure *by how much* the utility of the chosen action falls short of the best one, turning the landscape continuous and giving the optimizer a direction to descend; `margin` additionally demands a safety gap, rewarding decisions that win *clearly* rather than by a little. We test them to see whether this smoother signal accelerates or stabilizes convergence relative to `binary`.
- **`softmax`** and **`entropy`**. These score the log-likelihood of the expert action under a softmax over the candidate utilities; `entropy` adds a regularizer that discour-

ages over-confident utility vectors. We test them to check whether the confidence level improves the recovered model.

All five will later be crossed against the optimizers; the preliminary study below will fix `binary` because it is the simplest and the easiest to evaluate.

5.3 Convergence and the stopping criterion

Prior to conducting a full sweep of the hyperparameter and configuration grid, we performed a couple of preliminary studies. This first study aims to solve, or at least hint at the solutions for, two linked questions that are kept fixed throughout the grid: *when a run should stop*, and *what does converge mean* in the context of a recovery problem.

With the `binary` fitness, an individual computes 0 penalty score when it reproduces *every* training rule it was shown; we call such individuals **satisfiers**. A natural stopping rule is therefore to wait until a given fraction of the population has become a satisfier: the `top`X criterion interrupts the run once $X\%$ of the individuals satisfy all the training rules. We track the thresholds $X \in \{50, 70, 90, 95, 100\}$ together with the four EDA variants (UMDA, EMNA, EGNA and KEDA on *bypass*; only the cheap UMDA and EMNA on the *NHLy*, where the multivariate EGNA and the kernel-based KEDA are too expensive for this preliminary study) and the `binary` fitness, exposing 5, 10, 20 and 30% of the decision-rule base.

The two benchmarks differ in what is recovered and in how many rules those percentages represent. Besides, on *bypass* the recovery optimizes *both* the conditional probability tables and the utilities (`both` mode) and each configuration is repeated 10 times; its rule base has 20 decision rules, so the four fractions correspond to 1, 2, 4 and 6 rules. On *NHLy*, whose size makes a full recovery expensive, *only* the utilities are optimized (`utility_only` mode, leaving the probability tables untouched) and each configuration is repeated 5 times; its 67 decision rules turn the same fractions into 3, 7, 13 and 20 rules.

Each run is capped at 100 generations with at least 50 so that the convergence curves show enough evolution, and each repetition draws a fresh random rule sample and initialization (different `seed` in random function).

The crucial point is that satisfying the training rules does *not* fix the complete optimal policy. Many individuals achieve a perfect, zero-loss, fit to the rules they were shown and yet disagree on the rules they were *not* shown (due to the degeneracy explained in Chapter 3). To expose it we measure their accuracy on the **whole** rule base, and report its mean and its standard deviation across the repetitions.

5.3.1 The stabilization level

Figure 5.2 shows ten repetitions of a single configuration (UMDA, `binary`, 30% of the rules) together with their mean. **Every repetition stabilizes** (as the nature of an EDA guarantees): after the first generations each trajectory reaches a plateau and stays on it, so the method is well behaved on all seeds. What changes from one run to another is not *whether* it settles but the *level* at which it does. In this particular configuration most repetitions stabilize on a 75–85% plateau, but at least one settles

much lower, around 50% (not a failure to converge, but convergence to a policy no better than the one obtained with no rules at all) so the across-repetition mean sits around 72%, below the better individual runs.

Because each repetition starts from a different random initialization and sees a different rule sample, **the outcome of a single run is itself a random variable**: the same optimizer, fitness and rule budget can stabilize on a good policy or on a mediocre one depending only on where it started. The markers `top50/70/90/95/100` on the mean curve give the median generation at which each stopping threshold is reached; that they cluster within a narrow band of generations already anticipates the next result.

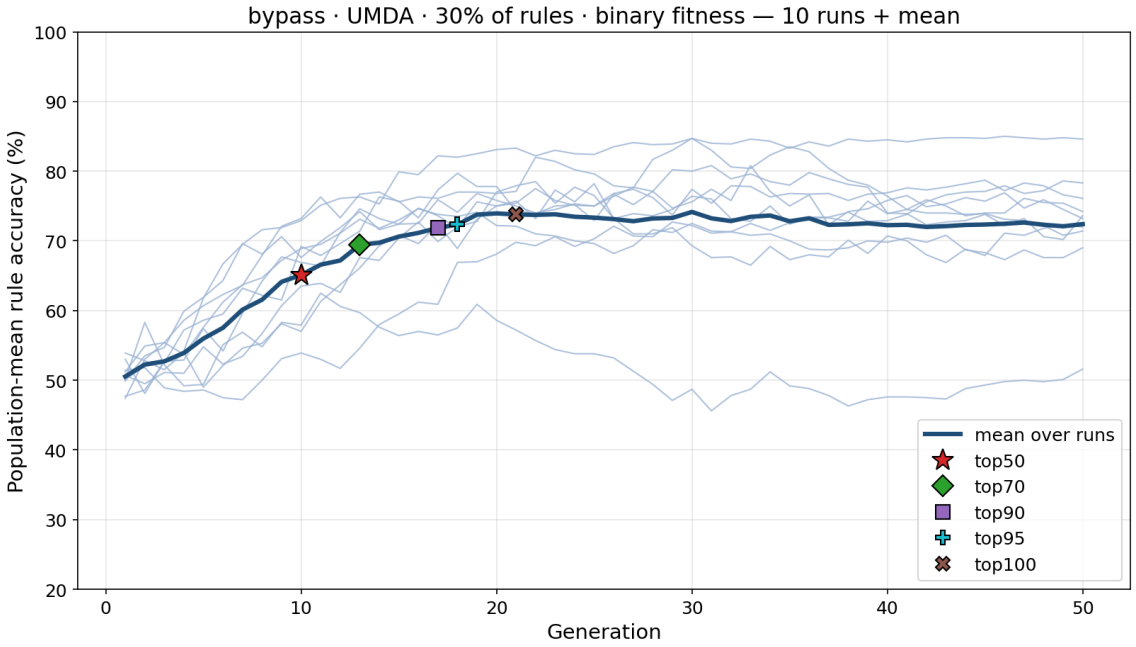

Figure 5.2: Ten repetitions of a single configuration on *bypass* (UMDA, `binary`, 30% of the rules). Thin curves: individual repetitions, each with its own random initialization and rule sample. Thick curve: across-repetition mean. The markers `topX` indicate the median generation at which the corresponding stopping threshold is reached.

5.3.2 The stopping threshold: accuracy versus cost

The stopping criterion controls *when* to stop, and therefore also the cost of a run. It does not change *what* the search does (with a fixed seed the EDA visits the same sequence of populations). The question then is purely one of trade-off: *how much accuracy a later stop will yield the extra generations it costs*. Figure 5.3 makes it visible on one *NHLy* configuration.

Convergence is slow, so the curve keeps climbing well beyond `top50`: a stricter threshold genuinely buys accuracy, but every increment costs many generations of an already expensive run, and the strictest thresholds are often *never reached* within a reasonable budget (the `top100` marker is absent). A single configuration cannot fix the criterion, though; the trade-off has to be read across all of them at once.

Figure 5.4 does exactly that. Setting the threshold from 10% to 100% and averaging over every optimizer, rule fraction and repetition, it plots the recovered accuracy

against the cost (generations) at which each threshold would fire. The curve has a very clear shape on both IDs: accuracy climbs fast, bends at a knee, then grows significantly slower. **We adopt `top90`** as that knee (the threshold that captures essentially all the achievable accuracy while remaining attainable within a reasonable budget.

Going further is a not worth enough: `top95` and `top100` add at most a point before flattening, while the cost climbs from $\sim$14 to $\sim$25 generations, and `top100` is reached in only about two-thirds of the runs (66% on *bypass*, 60% on *NHLy*). Stopping earlier (`top50`) would instead forgo several points specially on *NHLy*. `top90` is thus the strictest threshold that is still robustly reachable and we fix it for the rest of the study.

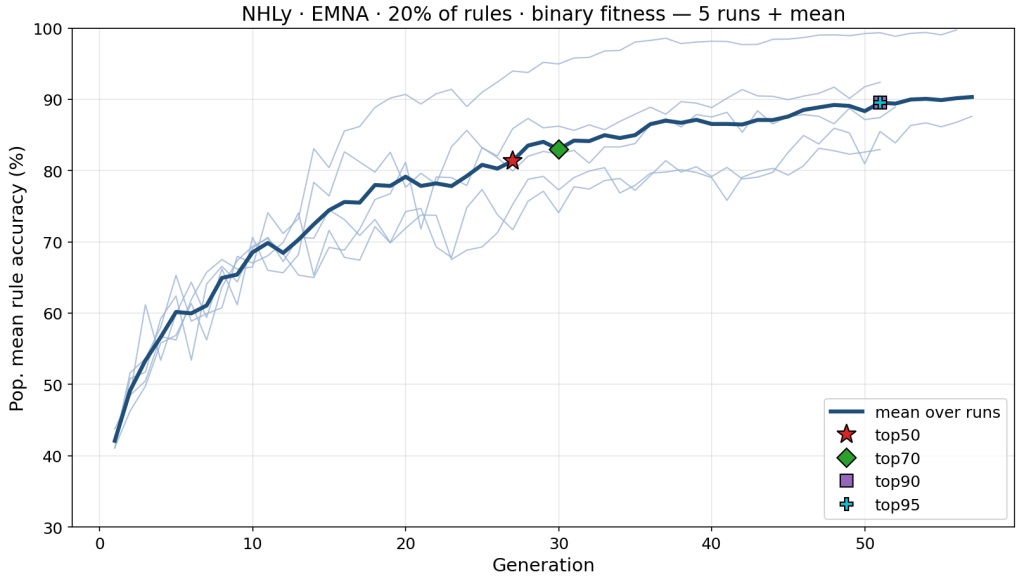

Figure 5.3: Convergence of (*NHLy*, EMNA, `binary`, 20% of the rules) configuration; same conventions as Figure 5.2. Convergence is slower (still rising at `top50`).

Two reasons allow us to reuse this single criterion throughout all the experiments, even across completely different configurations such as varying loss functions.

- First, `top90` is not tied to the `binary` notion of a satisfier. Instead, it triggers when "the best 90% of the population reach the *target quality* of the loss in use". This target is defined per loss function (e.g., zero penalty for `binary`, the prescribed margin for `regret`/`margin`, and the corresponding log-likelihood threshold for `softmax`/`entropy`, as detailed later).
- Second, the fitness curve shape that justifies this choice (a steep rise followed by a plateau) is an inherent property of the *EDA dynamics* on this kind of problem, rather than an artifact of any particular loss function. Although the length and steepness of this curve may vary, the loss merely reshapes the landscape that the search algorithm climbs.

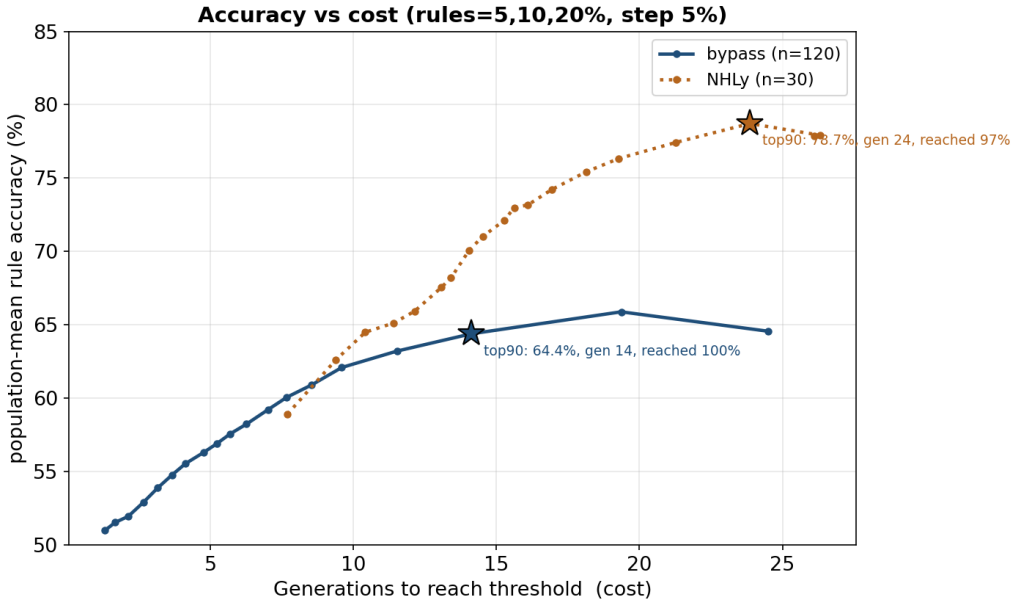

Figure 5.4: Accuracy versus cost as the stopping threshold sweeps $10\% \rightarrow 100\%$ in steps of 5%, averaged over every optimizer and repetition for 5, 10 and 20% of the rules (population-mean rule accuracy, `binary` fitness). The x axis is the generation at which each threshold fires, its cost. The stars mark the adopted `top90` and report its accuracy, cost and how often it is reached.

5.4 Population size of the EDA search

Before moving to the main results, we must fix one last parameter: the **EDA population size**. This is crucial because it determines *how reproducible* a single run is, rather than *whether* it succeeds.

To set this, we conduct a variance study on *bypass* (`both`) and *NHLy* (`utility_only`). We sweep the population size across several sizes. For each size, we run a full grid of combinations: 5 rule subsets $\times$ 5 initialization seeds over the 5 losses.

Because the rule sample and the population initialization are driven by *independent* seeds, the accuracy variance splits into two components (each caused by a separate stochastic draw): the **rule effect** (how much the result moves when the elicited rules change) and the **init effect** (how much it moves when only the random initialization of the parameter vector changes).

5.4.1 Larger population yields stability, not accuracy

Looking at the *bypass* sweep along the cost axis (Figure 5.5, right panel) mean accuracy remains *flat*: it sits at $\sim$67–72% for UMDA/EMNA/EGNA and $\sim$78–80% for KEDA almost regardless of the population, so a larger population does *not* recover a better policy. What the population changes is the *dispersion* (left panel). The **init effect falls sharply** as the number of individuals grow. At the same time the **rule variance stays roughly flat** ($\sim$2–3 points) throughout the sweep.

This is the central finding of the section: the population suppresses the spread that comes from the stochastic initialization of the optimizer, but it has a weaker effect on

the variance that comes from *which questions the expert is asked*, which depends on the data input, not of the search. EMNA is the exception its init effect barely falls (it stays around 3 points even at 400), a structural instability the population in not solve (and the cost, of course, rises with the population).

On *bypass*, a cheap network of seconds per run, the init effect for UMDA does not improve up to 200 (it is still $\sim$3.4, as at 50) and only collapses at **400** ($\sim$1.1); pushing on to 800 buys a marginal 0.2 point for double the cost. We have chosen 400 as it provides a good trade-off, and that is the population we adopt for *bypass* throughout the rest of the chapter.

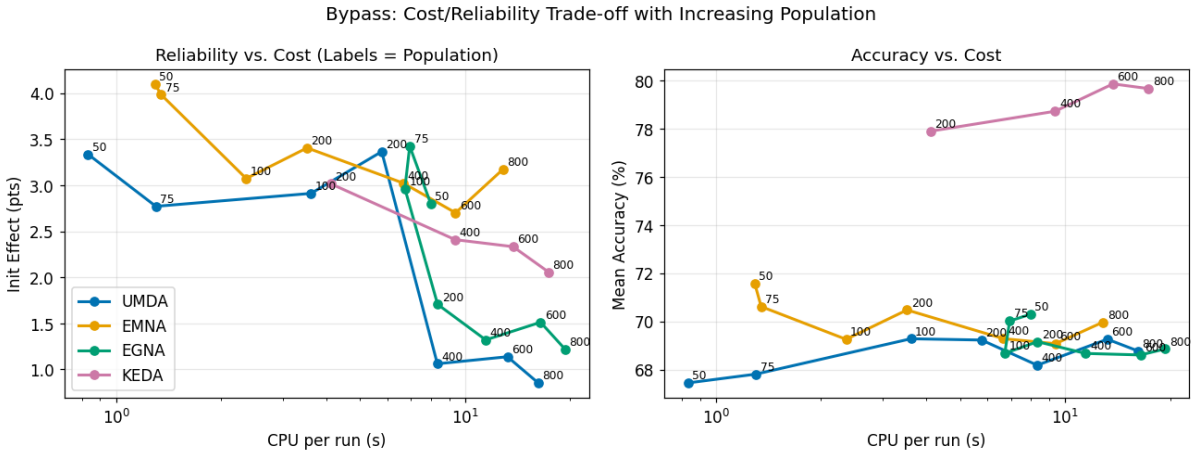

Figure 5.5: *Bypass* (`both`): init effect (left) and mean accuracy (right) against CPU cost per run, with the population labeling each point.

5.4.2 The population drains the init variance, not the rule variance

The flat rule effect and the collapsing init effect can be read directly by plotting one component against the other (Figure 5.6). Points above the diagonal are configurations where the initialization dominates the result; points below are where the drawn rules dominate. At population 100 most configurations sit *above* the diagonal: the initialization is still the larger source of variance. At a population of 400, the same points have dropped *below* the diagonal (the init. variance is reduced to $\sim$1 point, while the rule variance remains unchanged at $\sim$2–3), leaving only the irreducible sensitivity to the rule sample. This is precisely the behavior anticipated in the previous section, providing strong justification for selecting a population of 400 for the *bypass* model. Ultimately, we set a population size where the spread introduced by the initialization becomes negligible compared to the inherent variance of the data.

5.4.3 Population effect on NHLy

The population size is chosen *per network*, and *NHLy* presents a great contrast to *bypass* (Figures 5.7 and 5.8). At a population of 50, the recovery is already far more stable: both variance components sit low (UMDA clusters near the origin, with both effects around a single point) because `utility_only` is a much more constrained problem with little variance left to remove. Crucially, the plot (Figure 5.8) shows that most configurations are *already below* the diagonal at 50 meaning the rule sample,

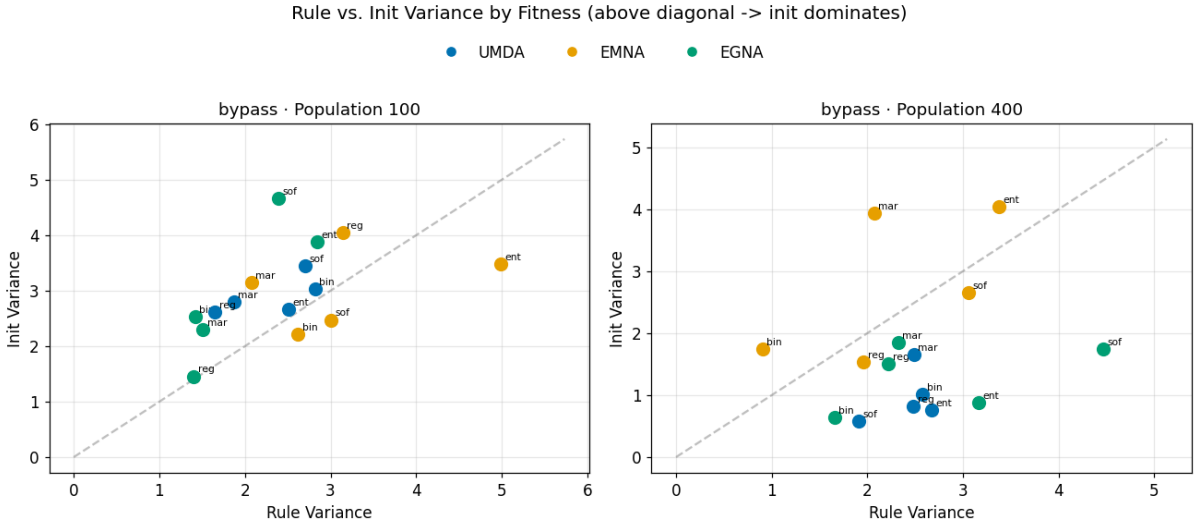

Figure 5.6: *Bypass*: rule effect versus init effect per fitness and optimizer, at population 100 (left) and 400 (right). Above the diagonal the initialization dominates; below it the drawn rules do. Growing the population from 100 to 400 moves the cloud from above to below the diagonal. The init variance is drained, the rule variance is not.

not the initialization, is the dominant factor. This is precisely the state that *bypass* only reached at a population of 400.

Because it is already below the diagonal, any population above 50 is simply unnecessary. Sweeping the population from 50 to 200 confirms that increasing the size has only a marginal effect (Figure 5.7). Given that *NHLy* is a highly expensive network (running at near to two orders of magnitude more cost per run than *bypass*) pushing beyond 50 is completely unjustifiable. Therefore, *NHLy* is run at **50**.

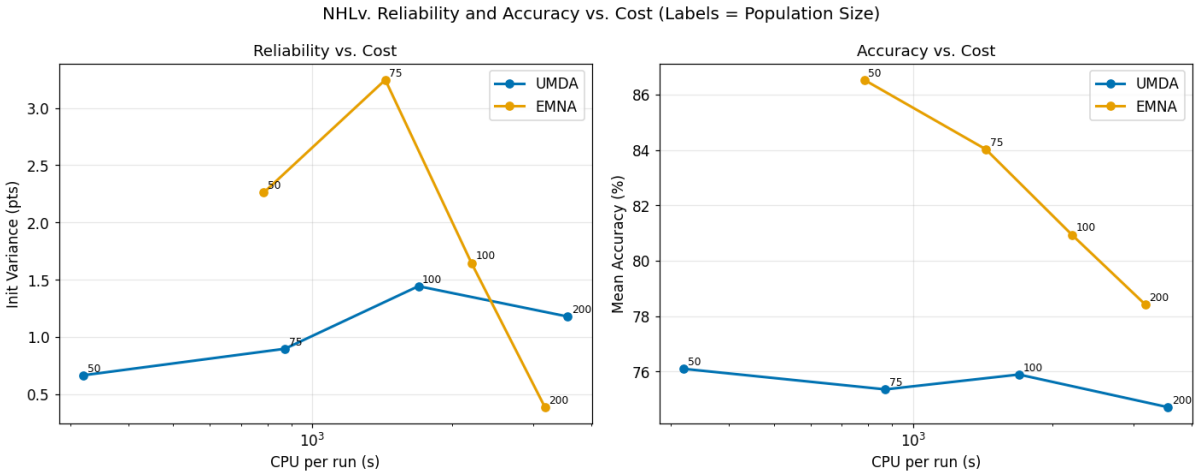

Figure 5.7: *NHLy* (`utility_only`, `binary+regret`): init variance (left) and mean accuracy (right) against CPU cost per run, population labelling each point.

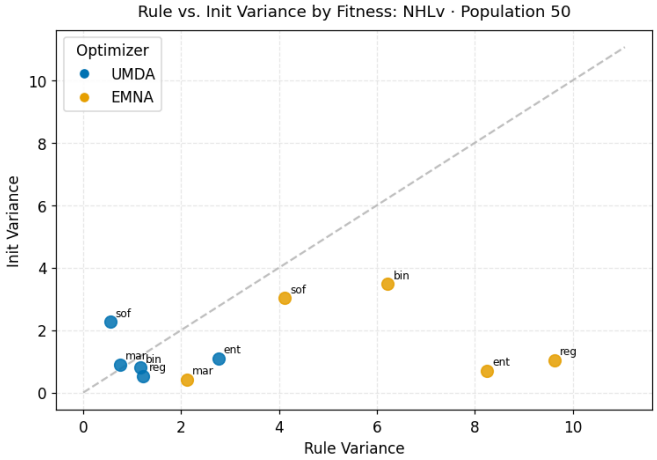

Figure 5.8: *NHLy* (`utility_only`, population 50): rule effect versus init effect per fitness and optimizer.

5.5 First results

Although the convergence study in Section 5.3 was conducted to determine the stopping criterion, it also provides the first relevant results of this chapter, detailed in Table 5.4. A key finding is related to the satisfiers (individuals that reproduce the training rules *perfectly*) because they exhibit completely different behavior in unseen scenarios. The convergence grids in Figures 5.9 and 5.10 demonstrate this progression across generations, where the gap between the "best" (green) and "worst" (red) individuals exemplifies this degeneracy.

Exploring this variance represents a promising line of research: it would be highly beneficial to identify the minimal set of rules that yields maximum accuracy and which combination of them provides the greatest information gain. However, although analytical methods exist for this, the present thesis will assume that a domain expert already knows, or at least has a hint, based on experience, which specific subset of rules guarantees a high success rate.

		mean accuracy (%)				**std (%)**			
net	**opt**	**5**	**10**	**20**	**30**	**5**	**10**	**20**	**30**
bypass	UMDA	54	59	64	71	3.8	3.6	2.9	4.6
	EMNA	54	59	64	73	4.6	5.2	5.9	6.5
	EGNA	56	63	64	71	3.5	5.6	3.5	5.4
	KEDA	73	73	77	81	4.6	5.0	7.1	7.2
NHLy	EMNA	81	80	89	94	2.2	2.1	6.6	6.3
	UMDA	73	75	79	83	1.9	2.5	2.7	3.6

Table 5.4: Accuracy on different rule sets of the satisfiers (indiv. that reproduce every training rule), by network, optimizer and fraction of rules exposed (5–30%), measured at the `top90` stopping point (Section 5.3.2). "Mean" is the satisfier accuracy averaged over the repetitions; "std" is its standard deviation *across* those repetitions. EGNA and KEDA were run only on *bypass*. On *bypass* all optimizers use the 400-individual population adopted in Section 5.4; *NHLy* uses 50.

5.5.1 Effect of the number of rules exposed

This subsection analyses how the amount of supervision, i.e. the fraction of rules shown to the search, affects the recovered policy.

- The first quantitative result is that **the recovery is clearly beneficial**. On every network and optimizer the mean satisfier accuracy in Table 5.4 stops well above the random baseline (50% on *bypass*, 42% on *NHLy*) and climbs steadily as more rules are exposed. Even the smallest budget helps, though modestly for the hardest optimizers: a *single* rule on *bypass* (5% of the base) lifts the mean to 54–73% — only a couple of points over the one-rule baseline ($52.5\% = \frac{50\% \times 19 test + 100\% \times 1 train}{20}$ on *bypass*) for UMDA/EMNA, but well above it for KEDA — and richer budgets raise it into the 71–94% range. At the larger budgets, then, the recovery buys up to twenty-odd accuracy points at no extra cost, merely through the application of EDAs.
- The second message is the other side of working with so few rules: *which* rule is drawn matters. Each repetition sees a different random rule subset, so the standard deviation in the table measures the sensitivity to that draw. It stays within a few points, but with a single rule a bad draw can still pull a run down to the baseline (notice that a rule will always contributes to improve the accuracy and if a run falls below the based line is only a matter of probability).

The number is relevant, but also the quality of the rules that support the estimation of the parameters through the EDA, the generic rules, the exception cases, and the coverage of the problem; it is the expert who evaluates this aspect of quality.

5.5.2 The role of the optimizer

The optimizers differ in the level they reach, in their robustness to the rule drawn, and in their cost (Figure 5.9). **KEDA is both the strongest and the most robust** on *bypass*: it reaches the highest mean (69–81%) and the smallest sensitivity to the rule drawn (std ≤ 6; even its worst repetitions stay around 65%), not because of the population number (400 individuals have been also tried in UMDA and EMNA with t) but due to its kernel-density model that covers all the search space, finding better solutions.

UMDA, EMNA and EGNA reach a similar, lower level ($\sim$ 55–76%). Cost, however, separates them on two compounding counts. Per generation, UMDA and EMNA are about an order of magnitude cheaper than EGNA and KEDA (~ 0.05 against ~ 0.55 CPU seconds). And the expensive pair also needs *more* generations to reach `top90` and stabilize (KEDA a median of 19 generations against UMDA's 12) so the two factors multiply: reaching the stopping point costs about 0.5 s with UMDA or EMNA but 8–10 s with EGNA or KEDA, a 15–$20\times$ gap (Figure 5.11). This makes **EGNA the poorest in cost–benefit**: as slow as KEDA, yet no more accurate than the cheap UMDA and EMNA.

On *NHLy* only the cheap pair is affordable: EGNA and KEDA are dropped, as their per-run cost is prohibitive on a parameter vector this large (and KEDA's kernel estimate degenerates under certain conditions). Here the EMNA-versus-UMDA contrast becomes decisive (Figure 5.10).

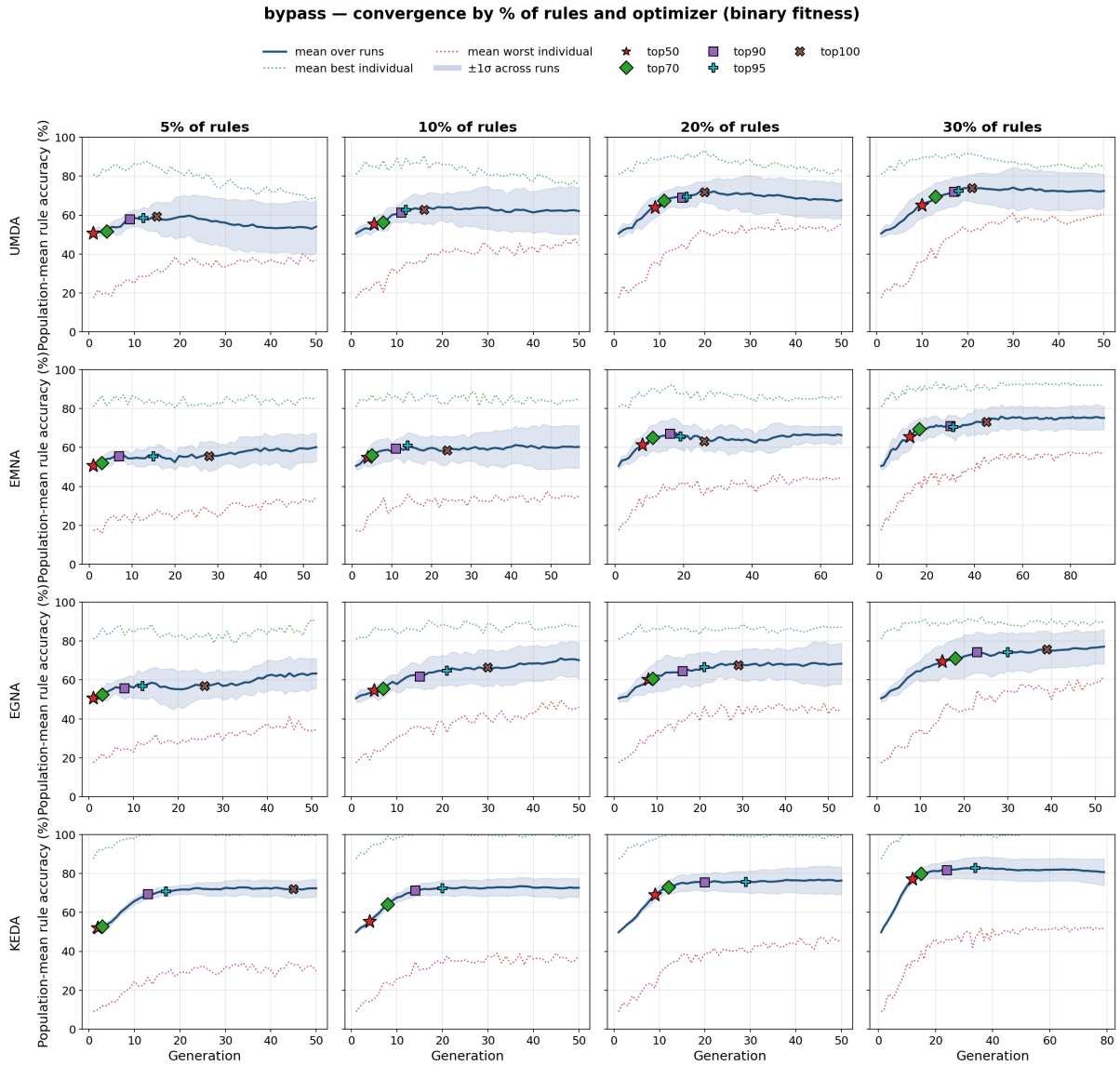

Figure 5.9: Convergence on *bypass* by fraction of rules (columns) and optimizer (rows: UMDA, EMNA, EGNA, KEDA), `binary` fitness. Solid: mean population accuracy with a $\pm 1\sigma$ band across repetitions; dotted: mean of the best (green) and the worst (red) individual; markers: the `top50`–`top100` transitions.

EMNA clearly wins: its mean climbs from 81% to 94% as rules are added, whereas **UMDA** is limited in a lower 73–83% band (although both sit far above the 42% baseline and both are stable across rule draws, std $\lesssim 7$). The grid shows the reason UMDA population collapses early into a tight but low region, while EMNA keeps exploring until it reaches the high-accuracy one. That extra accuracy is paid in convergence time, not in per-generation speed (Figure 5.11). At small-to-moderate budgets EMNA needs more generations to reach `top90` (a median of 18–51 against UMDA's 8–19) so its recovery costs roughly twice as much.

Figure 5.10: Convergence on *NHLy* by fraction of rules (columns) and optimizer (rows: UMDA, EMNA), `binary` fitness, `utility_only` mode. Conventions as in Figure 5.9.

5.5.3 Convergence dynamics and what is worth recovering

Two contrasting trends emerge from the convergence grids (Figures 5.9 and 5.10). Across repetitions, the $\pm 1\sigma$ band **widens** (in the *bypass* network at 30% of the rules, it increases from ~ 2 to ~ 6–10 points before saturating). On the opposite, *within* a single run, the gap between the best and worst individuals **narrows** (in *NHLy*, decreasing from ~ 74 to ~ 5 points). Viewed as a Markov chain, each repetition represents an independent path that originates from a near-uniform model and drifts toward a *distinct* satisfier optimum (Section 5.5).

The inter-run variance accumulates similar to a random walk, eventually plateauing at a width that reflects the extent of the degenerate subspace, while selective pressure continuously contracts each population toward its specific point of convergence. Consequently, the individual runs diverge from *one another*, even as they become internally more homogeneous.

A final practical observation merits attention. In the *bypass* configuration, the recovery process fits both the probability tables and the utilities (`both`), whereas in *NHLy*, it only recovers the utilities (`utility_only`). Despite its significantly larger size, *NHLy* achieves higher accuracy ($\sim$73–94% compared to $\sim$55–81%). This indicates that **attempting to recover the probabilities is detrimental rather than beneficial** (at least with this binary fitness function); the rules fail to identify them, and the introduction of extra free parameters merely expands an already degenerate search space.

We draw two primary conclusions from this study:

- The stopping criterion behaves as a knee point characterized by decreasing returns, justifying our selection of `top90`.
- *Reproducing the training rules is not equivalent to recovering the underlying policy.* The only reliable method to bridge this gap is to expose the system to a broader

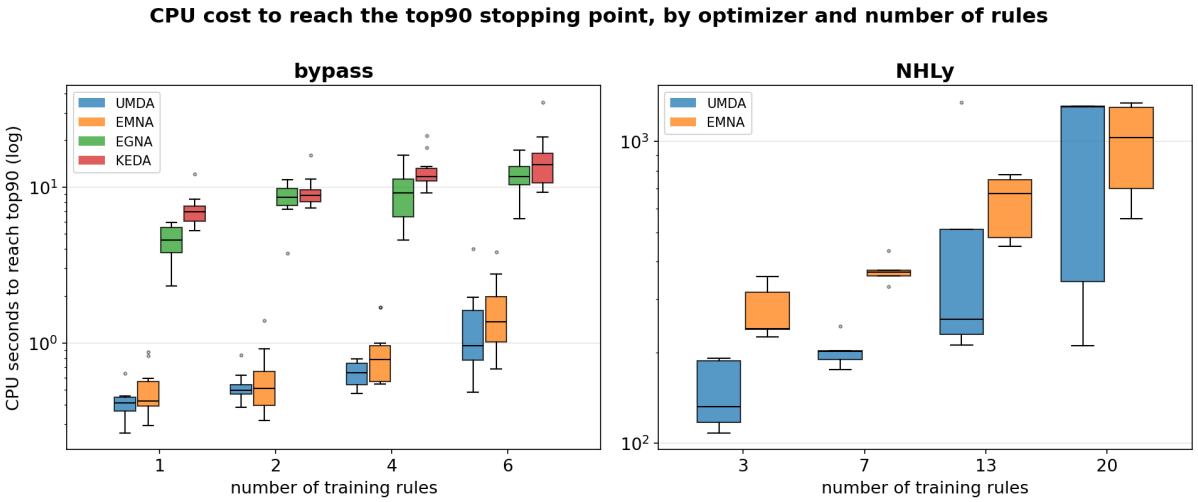

Figure 5.11: CPU seconds to reach the `top90` stopping point (per run, logarithmic scale), by optimizer and number of training rules on the tho IDs.

set of rules.

5.6 Comparing the loss functions

While the convergence study (Section 5.3) established *when* a run should stop, and the population study (Section 5.4) determined the appropriate number of individuals, this section evaluates the impact of using different loss functions for the recovery process. We compare the five loss variants introduced in Section 5.2.3 across two main configurations. The first is the *bypass* case in the joint `both` mode (population of 400, 20% of the rules exposed), and the second is the *NHLy* case in `utility_only` mode (population of 50). For both setups, we perform five independent repetitions per loss function and optimizer. We evaluate each recovery based on its mean population accuracy and CPU time. Ultimately, the goal is to determine whether a smoother, more informative loss function recovers the policy more effectively than the plain `binary` baseline, or if the added complexity yields no tangible benefit.

5.6.1 Bypass

On *bypass* the five losses are **interchangeable in accuracy** (Table 5.5): the spread across losses and optimizers is smaller than spread over the repetition. The smoother landscape the continuous and likelihood losses promise does not translate into a better recovered policy.

	acc_mean (%)		**gens.**
loss	UMDA	EMNA	(range)
`binary`	66	63	6–16
`regret`	65	65	5–16
`margin`	67	66	19–24
`softmax`	66	64	9–15
`entropy`	66	68	9–15

Table 5.5: *Bypass*, `both` mode, population 400, 20% of the rules, five repetitions. "acc_mean" is the mean population accuracy per optimizer; "gens." the range of stopping generations across the two optimizers.

Because the accuracies coincide, the choice falls to secondary qualities. Here the losses do differ: `binary` and `regret` converge fastest and `margin` slowest, and `binary` is also the simplest and cheapest to evaluate. As accurate as any, among the quickest to converge, and the least complex, `binary` is the natural baseline on this network.

5.6.2 NHLy

The results on the larger *NHLy* dataset, demonstrate that the choice of loss function becomes decisive when the search space requires a more active guidance (Table 5.6). Furthermore, performance is highly tied to the optimizer: the continuous, utility-aware `regret` loss clearly maximizes EMNA's capabilities, reaching a peak accuracy of 89.7%, followed by the `binary` baseline at 83.9%. The remaining losses fail to provide an advantage for EMNA. Conversely, UMDA is unable to exploit any of the loss formulations; as noted in Section 5.5, it collapses early and stagnates between 75% and 78% regardless of the chosen metric.

loss	UMDA	EMNA
`regret`	75.1	**89.7**
`binary`	75.2	83.9
`entropy`	77.4	78.7
`softmax`	78.1	77.5
`margin`	76.4	77.5

Table 5.6: *NHLy*, `utility_only`, population 50, 10% of the rules, mean population accuracy (%) averaged over repetitions.

5.6.3 The utility temperature shapes the recovery

To control the effect that the decoding of the parameter vector may have on the loss space across the different fitness functions, we sweep across five utility temperature (T_u) values. In essence, this temperature modifies the utility values to either smooth or accentuate the resulting utility distribution. This effect can soften or sharpen the optimization landscape, preventing the search from saturating prematurely or getting trapped in suboptimal regions.

To isolate its effect, we fix the true CPTs and recover only the utilities on the *bypass* configuration (`utility_only`, population 400), sweeping T_u across a geometric range centered around 1. Figure 5.12 illustrates the recovery performance (both in terms of final accuracy and convergence speed) across the different losses.

Two clear, monotonic trends emerge, specifically for the stricter `binary` and `regret` losses. As T_u increases, it simultaneously **improves the mean accuracy** (e.g., `binary` climbs from $\sim 95\%$ at $T_u = 0.25$ to $\sim 97\%$ at $T_u = 4$) and **speeds up convergence** (the required generations drop from 5.1 to 3.8). Note that we removed `margin` from the speed heatmap because it never satisfied the stopping criterion and consistently hit the 50-generation cap, so it is omitted from the convergence plot to preserve the color scale for the others.

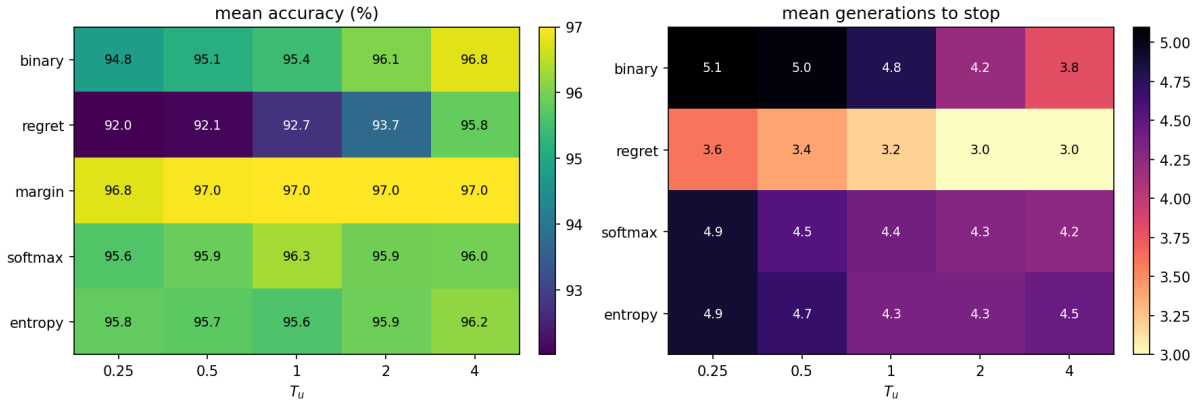

Figure 5.12: *Bypass*, `utility_only`, population 400, 20% of the rules, averaged over the two optimizers and repetitions. Left: mean population accuracy over the loss $\times\, T_u$ grid; right: mean generations to reach the `top90` stopping point.

The reason follows directly from how the parameters are decoded (Section 4.3): each utility is the sigmoid $\sigma(\phi/T_u)$ of a raw searched value, so T_u sets the steepness of that

squashing (sharp and saturating at low T_u, quasi-linear at high T_u). What is specific to this experiment is the effect of that steepness on an *EDA*, which samples a whole population of raw vectors around its current estimate. At a low T_u the saturation makes each decision fragile and instable: near a boundary an infinitesimal change of the raw vector flips the decoded utility between extremes and the search needs more generations to settle. At a high T_u the squashing is gentle, the same population spread leaves the decisions unchanged, the population decodes to near-identical policies, and the smoother landscape is reached sooner.

A low T_u saturates the decoded utilities to extreme values, which impacts each loss differently. This is our interpretation :

- **`regret`:** As a magnitude-based loss, it relies on continuous gradients. Under a low T_u, the utility gap collapses to either 0 or the maximum range, degenerating into a scaled `binary` loss. A higher T_u keeps the utilities graded and restores this gradient, which is why `regret` benefits the most.
- **`binary`:** This loss only evaluates the $\arg\max$ and, unlike the others, has no built-in mechanism to keep utilities away from extreme values. It is completely exposed to saturation. A higher T_u acts as a gentler decoder, pulling the utilities back into a graded, well-separated regime, which explains its improvement on *bypass*.

Meanwhile, `margin`, `softmax`, and `entropy` already sit at their accuracy ceiling in this scenario, leaving no room for further improvement.

However, these results may be specific to *bypass*. On a network with a different geometry the balance could well invert: the extreme utilities that a low T_u produces are also a way of sweeping the utility space more aggressively, and a harder problem (where the search must travel further to separate the actions) might benefit from that wider exploration rather than be penalized for it.

5.7 A few rules recover most of the policy

A recovery keeps improving as more rules are shown (Section 5.5); this section asks the natural question "*how many is enough*". We measure it with the following experiment: for each rule budget k we launch an *independent* recovery that is shown exactly k true rules and let it *run to convergence*; the resulting model is then scored against the *whole* policy. Sweeping k traces how far the recovery reaches as a function of how many rules it is given (each k a fresh run). On *bypass* we sweep $k = 1, 2, \ldots, 20$ one rule at a time (three repetitions, each with a different set of training rules and its own initialization) at the 400-individual population fixed in Section 5.4, in `both` mode (recovering the chance tables *and* the utilities) and with each of the two fitness functions, `binary` and `regret`.

Crucially, and unlike the earlier sections, each run is now let converge rather than stopped early: it runs for a minimum of 20 generations (or earlier if *all* of the population reproduce the rules, `top100`) and then stops as soon as the best 90% do (`top90`), capped at 100 generations. The minimum matters because, when a high fraction of the rules is used for training, the `top90` criterion is met too quickly (gen. 1 or 2) and, on its own, would halt the search before the model has had time to reach accuracies close to 100%. On the much larger *NHLy*, where each recovery is far more expensive, we sweep $k = 3, 6, \ldots, 21$ in steps of three, with three repetitions, the `regret` loss stopped at `top70` (here we do not reach high fraction of rules during training so we can use a looser criterion), in `utility_only` mode. Both networks use UMDA and EMNA.

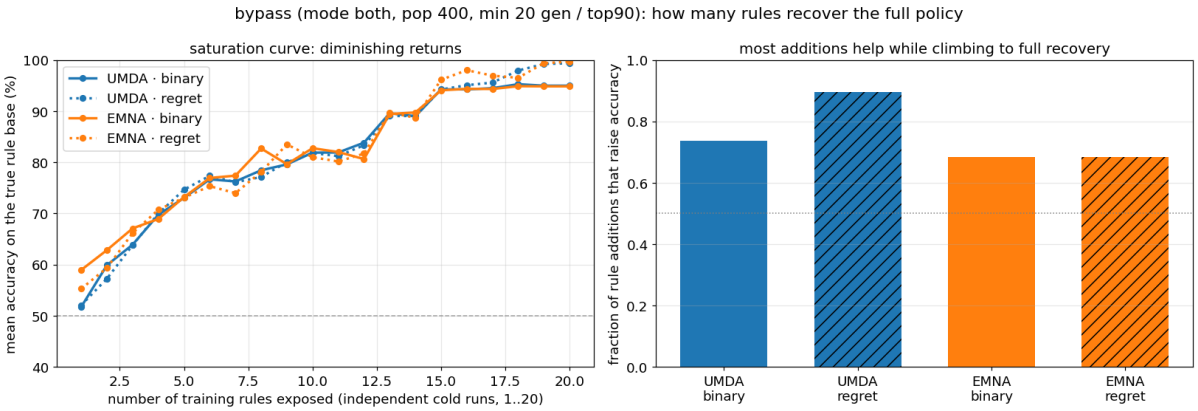

Figure 5.13: *Bypass* (`both` mode, population 400, min. 20 generations then `top90`): capacity sweep over the number of exposed rules ($k = 1, \ldots, 20$, independent recoveries, three repetitions), with `binary` (solid) and `regret` (dotted). Left: mean accuracy on the true rule base against the number of rules exposed. Right: the fraction of one-rule additions that raise accuracy.

Two things emerge on *bypass* (Figure 5.13). First, the recovery gain is **clearly nonlinear** with respect to the number of rules. An untrained model averages $\sim$50% (pure chance on the binary decisions) and a *single* rule barely moves it ($\sim$ 52% for UMDA, $\sim$59% for EMNA): one constraint is far too little to pin the whole diagram. From there, the curve grows steeply as the first handful of rules buy the most. Accuracy climbs to $\sim$ 73% by five rules and $\sim$ 82% by ten, capturing roughly two-thirds of the total gain. After this rapid initial growth, the curve bends and flattens out, demonstrating

that achieving 100% compliance requires observing a very high percentage of the total rules.

Second, the picture **barely depends on the loss or the optimizer**. UMDA and EMNA track each other almost exactly across the sweep (EMNA is only ahead at the very first rules: 59 vs 52% at $k = 1$) because *bypass* is small enough for the independent-marginal model of UMDA to capture. The loss separates them only at the very end: `regret` pins the policy more completely, closing to $\sim 100\%$ at twenty rules, while `binary` plateaus around 95% (`top100` is not achieved within the 20 generations in *bypass*). Here **most added rules do help**: the fraction of one-rule additions that raise accuracy stays well above one half for every configuration (0.68–0.89; Figure 5.13, right).

That parity breaks on the larger network (Figure 5.14). **EMNA converts rules into accuracy far better than UMDA**: it climbs steadily from $\sim 64\%$ at three rules to $\sim 95\%$ at twenty-one, while UMDA does more slowly and reaches around 79% (the same result seen in Section 5.5). Note also that here the coarse addition helps that the addition of three rules per step results useful. Each jump carries enough new signal that the accuracy keeps rising monotonically across the whole sweep except in only one case.

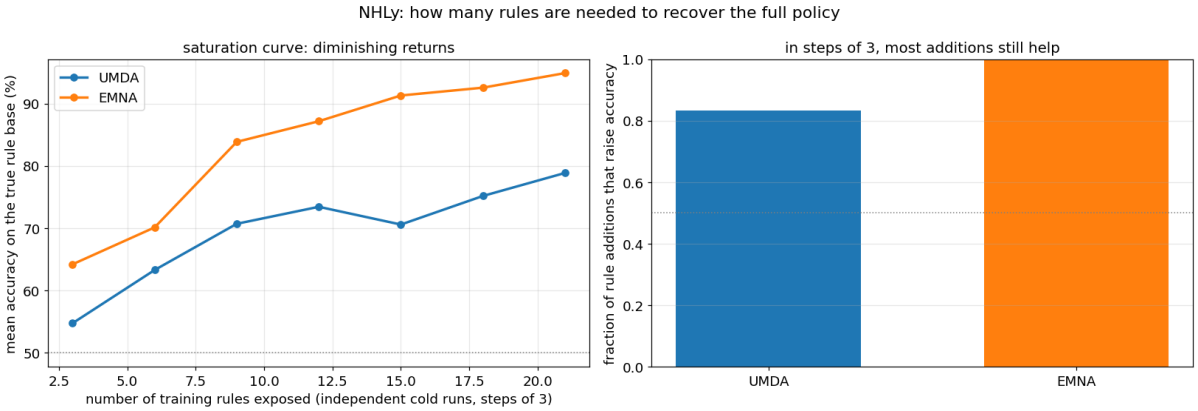

Figure 5.14: *NHLy*: the same capacity sweep on the larger network, coarser ($k = 3, 6, \ldots, 21$ in steps of three, three repetitions). Left: EMNA converts rules into accuracy steadily up to $\sim 95\%$ while UMDA lags and tops out lower. Right: at three rules per step every addition helps (100% for both optimizers), so the accuracy rises monotonically across the sweep.

The first few rules buy the largest gains, lifting the policy well above chance, while each further rule pays diminishing returns, and the last points up to full recovery are the expensive ones that need almost the whole rule base. Identifying *which* small set of rules is most informative rather than simply adding more.

5.8 Decisive rules matter more than their number

Section 5.7 showed that the first few rules capture most of the accuracy, but left open whether *which* specific rules are chosen actually matters. This section investigates exactly that: instead of adding *random* rules, we **search greedily** for highly informative ones and subsequently analyze what distinguishes a good rule set from a poor one. Starting from a single seed rule, at each step we evaluate *every* remaining rule as the next potential addition. We train each candidate set to `top90` (`binary` loss, `both` mode on *bypass*), and retain the one yielding the highest accuracy on the true 20-rule base (using a majority vote of the satisfiers, as in Section 5.3.2). We grow this set until isolating the best *eight* rules.

To prevent bias from a single lucky seed, we repeated the entire procedure across three distinct initial rules and network initializations (these three *chains* are seeded here by rules 14, 3, and 0) using both UMDA and EMNA. Consequently, at every step, we record not just the greedy winner but the accuracy of *all* other candidates, capturing the full distribution of how much the "next" rule matters.

5.8.1 The composition matters as much as the count

The central finding is that **for a fixed initial rule, the specific choice of the complementary rules leads to a wide dispersion in accuracy** (Figures 5.15–5.17). At *every* step, the performance gap between the best and worst candidates drastically exceeds the median gain of adding a new rule (a massive spread of ~ 20–35 points versus a median advance of just ~ 4 points). Put differently, a single well-chosen rule is worth several mediocre ones.

The greedy path climbs steadily to 90–95% but *saturates around five to six rules*. Beyond this, new additions yield negligible gains, perfectly mirroring the diminishing returns established in Section 5.7. Conversely, the worst choice at each step stagnates near the initial baseline. This proves that counting rules is only half the story; *composition* is the other half. Finally, both optimizers agree on this overall trajectory, with EMNA only lagging briefly at initialization due to noisier full-covariance sampling before catching up entirely by the third rule.

Crucially, the choice of rule dictates far more than just accuracy: it drives also a great variance in both **convergence cost** (CPU time) and **probability-table MSE** (Figure 5.18). For instance, at a fixed rule count, training time can swing by more than four times purely based on *which* rule is added.

Furthermore, these costs are essentially *correlated* with accuracy ($|\rho| \lesssim 0.15$). This implies that a poorly chosen rule is a double threat: it can be simultaneously less accurate and slower to converge. Despite this variance, the probability-table error remains consistently small throughout. This confirms the decoupling between accuracy and probability-table reconstruction: accuracy improves by rearranging the policy encoded by the utilities, not by reconstructing the original CPTs.

Figure 5.15: *Chain 0* (seed rule 14), *bypass* (`both`): accuracy vs. number of training rules, UMDA (left) and EMNA (right). Band is the Q25–Q75 spread over *all* candidate rules at each step; the thick line is the greedy winner (best candidate), the dotted line the worst, and the dashed horizontal line the accuracy of the single seed rule.

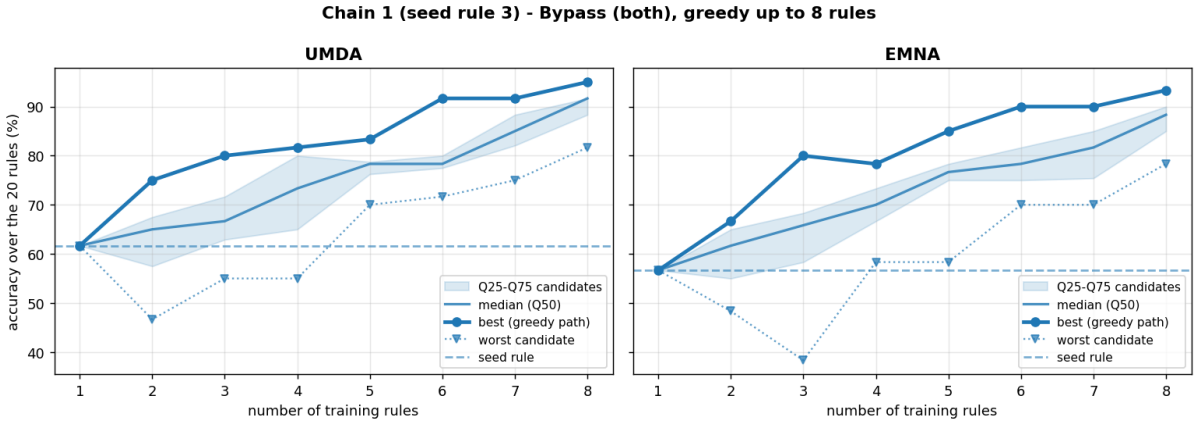

Figure 5.16: *Chain 1* (seed rule 3), *bypass* (`both`). Same layout as Figure 5.15.

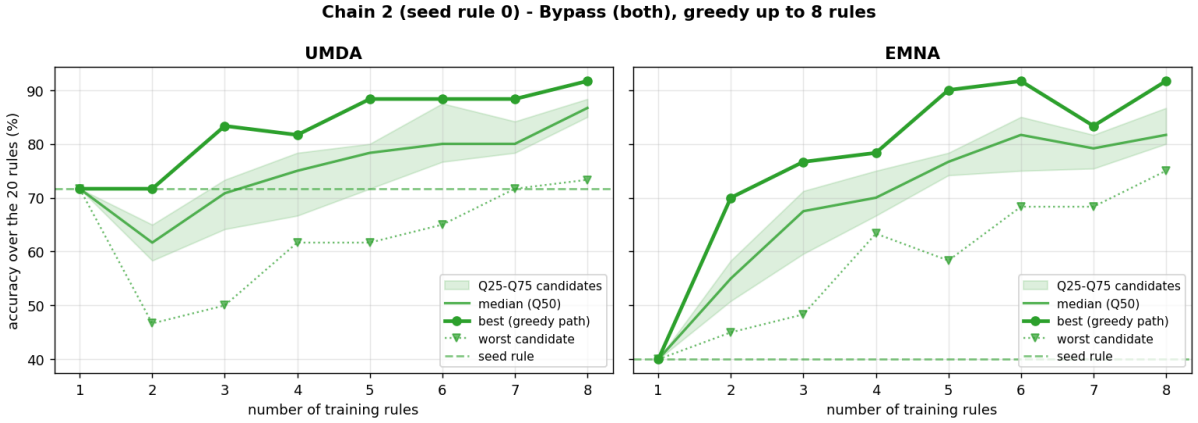

Figure 5.17: *Chain 2* (seed rule 0), *bypass* (`both`). Same layout as Figure 5.15.

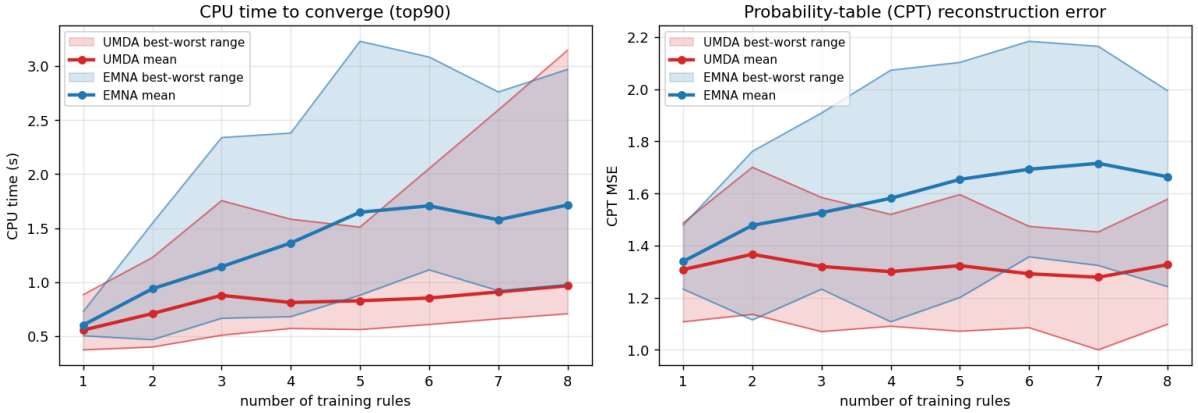

Figure 5.18: *Bypass* (`both`): best–worst **spread** over all candidate rules at each rule count (band between the fastest/lowest and slowest/highest candidate), aggregated over the three chains. UMDA in red, EMNA in blue. Left: CPU time to `top90`; right: probability-table (CPT) reconstruction error.

5.8.2 Decisive rules make the good sets

When mastering a medical domain, an expert intuitively internalizes the most critical, high-stakes decisions before addressing marginal or highly specific exceptions. We can quantify this learning hierarchy using a rule's *utility gap*: the expected-utility difference between the optimal action and its closest alternative, $|EU(\text{optimal}) - EU(\text{alternative})|$. In the *bypass* model, which chains two binary decisions (whether to medicate, `HEARTPHARMA`, and whether to operate, `HEARTSURGERY`), the wider this gap, the more decisively the scenario dictates the overall policy.

This is precisely the notion of informativeness that grounds regret-based utility elicitation: in the minimax-regret framework of Boutilier et al. [6, 7], the queries that most reduce uncertainty about the utility function are those with the largest utility gap, whereas near-tie scenarios (where every action yields almost the same expected utility) reveal little about the underlying preferences. We therefore advance the following **hypothesis**: a rule's utility gap is closely related to how strongly it drives the recovered policy, so that decisive, high-gap rules tend to be more informative than near-indifferent ones.

To probe this hypothesis we reuse the rule sets recovered in the previous experiment (`both` mode, population 400, `binary` loss, `top90`). From each of the three recovery groups we take the five highest-accuracy chains (fifteen rule sets in total) and count, for every rule, how many of these fifteen best sets contain it, discarding the first rule of each chain since it is drawn at random. Because the two decision nodes operate on very different utility scales (surgery triggers larger swings than medication), we keep them apart and plot each rule's appearance count against its utility gap, separated by decision node (Figure 5.19).

A correlation emerges between how often a rule appears among the highest-accuracy chains and its utility gap. In `HEARTSURGERY`, the appearance count climbs steeply with the gap: the two decisive rules (16, 18) dominate the best sets while the two near-indifferent ones (17, 19) never appear. In `HEARTPHARMA`, where the sixteen rules are narrowly clustered (0.09 to 0.46), the relationship is weaker, yet the most frequently

selected rules still tend to sit at the larger gaps.

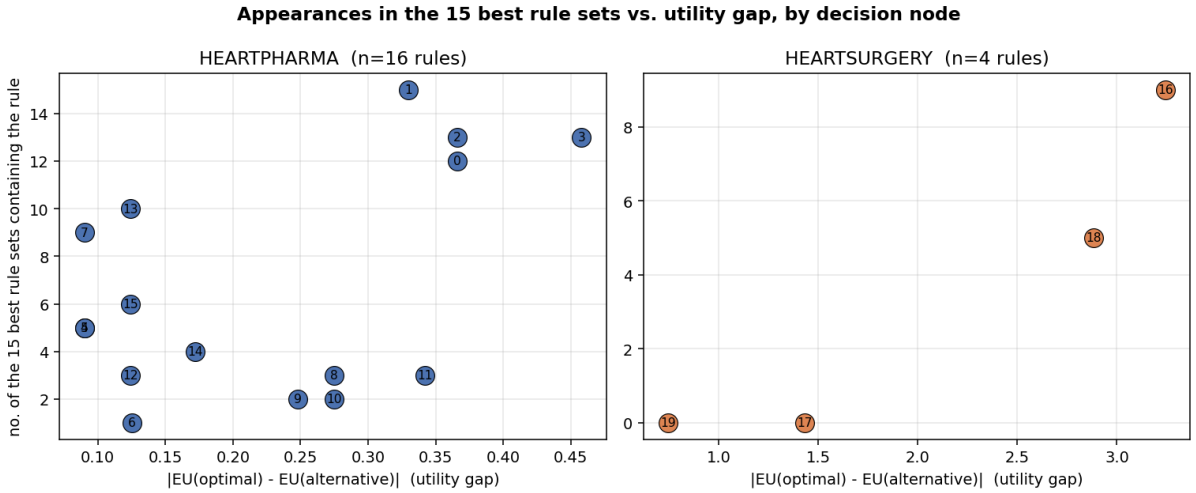

Figure 5.19: Appearances in the fifteen best rul*bypass* (`both`,population 400), split by decision node. Each point is a rule, placed at its utility gap $|EU(\text{optimal}) - EU(\text{alternative})|$; its height is the number of the fifteen highest-accuracy chains (the five best from each of the three recovery groups) that contain it, excluding the randomly chosen first rule of each chain. In the surgery node, where the gaps are widely dispersed, the decisive high-gap rules appear far more often; in the tightly clustered medication node the relationship is much weaker.

5.9 The Dominance of Utilities over Probabilities

As our final experiment, we now examine what the recovery process actually captures. The most straightforward test is to isolate each component of the model by fixing the other to its true value. We either search solely for the utilities (`utility_only`) or solely for the conditional probability tables (CPTs) (`chance_only`, while recording the CPT reconstruction error, the MSE), and then compare these isolated searches to a joint recovery of *both* components. Every run in this section uses the five loss functions, five initializations and five rule sets, at 20% of the rules and with the `top90` stopping rule. Our primary finding is that the utilities dictate the overall behavior: the policy is inherently driven by the utility matrix.

Figure 5.20 (left) illustrates the result of the each recovery. Searching exclusively for the utilities reproduces the policy with near perfection ($\sim$96% for both optimizers), whereas searching solely for the CPTs yields a much lower accuracy ($\sim$66%). By itself, this might not seem surprising, as the number of utility parameters is significantly lower than the number of parameters representing the CPTs.

Interestingly, however, recovering both components simultaneously (despite the increase in the total number of parameters) yields an accuracy comparable to, or even higher than, the `chance_only` baseline ($\sim$68%). This is further corroborated by our observations in Section 5.5: attempting to jointly recover probabilities and utilities in the *bypass* model results in lower accuracies than recovering a larger model like *NHLy* using only utilities, even though the parameter vector of the latter is less than half the size (64 versus 144, respectively, Table 5.1).

This counter-intuitive result can be explained by the degree of degeneracy inherent to each problem formulation. When probabilities are introduced as unknowns in the search space, degeneracy increases drastically due to the dense interdependencies among CPTs (an issue that barely affects utilities). This creates numerous degenerate solutions that satisfy the training rules but fail to generalize to all rules. At the same time, if only the CPTs are recovered while the utilities are kept fixed, the CPTs are forced to adjust erratically and drastically to fit those predefined utility values, resulting in slower and worse convergence (as demonstrated in our experiments). Conversely, if both CPTs and utilities are recovered, they can vary and mutually adapt at the same time, leading to a much more successful recovery of both seen and unseen rules.

Furthermore, accurately reproducing the decision rules does not guarantee a precise recovery of the CPTs. Across all `chance_only` runs, rule accuracy is **uncorrelated** with the CPTs reconstruction error (Figure 5.20, right; Spearman $\rho \simeq 0$). This indicates that a better alignment with the expert's rules does not necessarily translate into an improved estimation of the probability tables. As said, many other valid solutions with high rule accuracy appear.

Ultimately, we conclude that this discrepancy arises because the probability recovery problem is inherently more degenerate and complex than estimating the utilities. However, it should be noted that this has only been observed for the *bypass* case, and we cannot guarantee that it will always hold true universally.

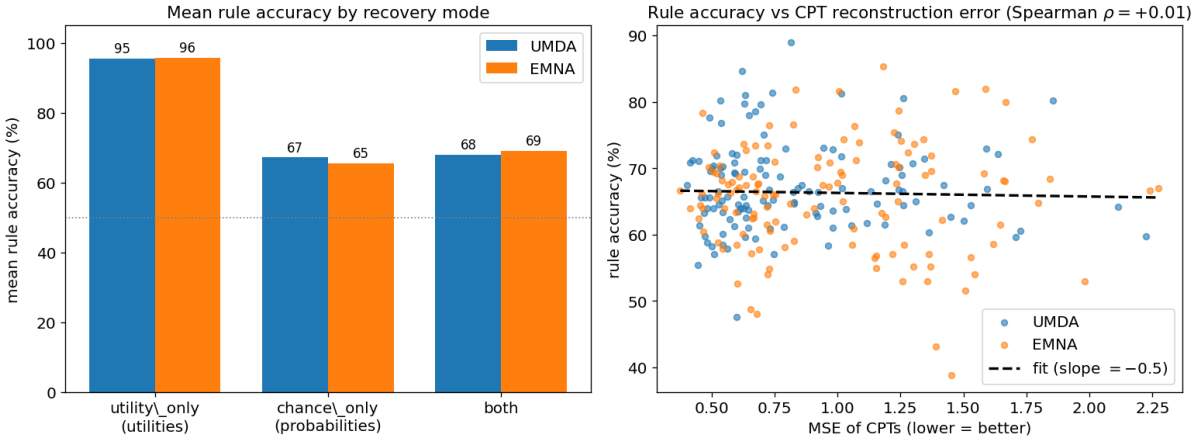

Figure 5.20: *Bypass* (20% of the rules). Left: mean rule accuracy when recovering only the utilities, only the probabilities, or both. Right: across all `chance_only` runs the accuracy is uncorrelated with the CPT-MSE.

Chapter 6

Conclusions and Future Work

This thesis studied whether the numerical parameters of an Influence Diagram (its utilities and conditional probability tables) can be recovered from a set of decision rules describing an expert's policy, using Estimation of Distribution Algorithms (EDAs). Chapter 3 posed the problem, Chapter 4 built the recovery pipeline, and Chapter 5 tested it on two clinical diagrams, the small *bypass* and the larger *NHLy*. This chapter states what was found and what remains open.

6.1 Evaluation of the objetives

1. **Recovery is feasible, and a small amount of supervision yields substantial reconstruction.** On every network and optimizer, showing only a fraction of the rules lifts the recovered policy well above the random baseline, and accuracy grows as more rules are added (Section 5.5).
2. **Reproducing the training rules is not the same as recovering the policy.** Many parameter vectors satisfy the shown rules perfectly yet disagree on the unseen ones: the problem is degenerate. This is the central finding, and the rest follow from it (Section 5.5).
3. **The optimizer is the dominant factor.** The choice of EDA matters more than any other setting. KEDA is the strongest and most robust on *bypass*, UMDA the weakest, but KEDA scales poorly and only the cheaper UMDA and EMNA remain practical on *NHLy*, where EMNA clearly outperforms UMDA (Sections 5.5 and 5.7).
4. **The loss election has low impact on the small network, but a utility-aware loss helps the large one.** On *bypass* the five losses are interchangeable in accuracy and the simple, cheap `binary` loss is never clearly beaten; on *NHLy* the continuous `regret` loss lifts EMNA markedly (to $\sim 90\%$). The utility temperature T_u further shapes the landscape and can be tuned to speed convergence (Sections 5.6 and 5.6.3).
5. **Which rules are shown matters as much as how many.** A few rules recover most of the policy and full recovery needs almost the whole base (Section 5.7); but for a fixed count the *choice* of rules drives a wide spread in accuracy, cost and reconstruction error, and the most informative rules are the *decisive* ones

(those with the largest expected-utility gap between the optimal action and its alternative, Section 5.8).

6. **What is recovered is the utilities, not the probabilities.** Searching only for the utilities reproduces the policy almost perfectly ($\sim 96\%$), whereas adding the conditional probability tables to the search makes the recovery substantially more complex: joint recovery falls back near the chance-only level ($\sim 68\%$), and rule accuracy is uncorrelated with how well the CPTs are reconstructed. The policy is carried by the utility structure, and introducing probability parameters only expands the dimensionality of an inherently degenerate search space (Section 5.9).

In short, the bottleneck is the *degeneracy* of the problem and the *expressiveness of the search*. Because the policy is carried by the utilities, recovery should target them while trying to recover the probabilities from actual databases or estimations; on top of that, the practical recipe is to pair an expressive optimizer with a simple `binary` loss on small diagrams (a utility-aware loss such as `regret` on larger ones) and to invest elicitation effort in a small set of decisive, informative rules.

6.2 Limitations

Only two diagrams were tested, so the exact figures need not carry over to much larger or different models. The rules are synthetic, drawn from a known ground-truth diagram; real elicitation would add expert biases (inconsistent, correlated and unevenly distributed rules) that this study does not model.

The estimates themselves rest on few samples. Each configuration was repeated only three, five or ten times, so the reported means and, above all, the standard deviations that we use as a reliability measure are noisy; they should be read as indicative trends rather than precise values.

On the large network the cost forced a further simplification: only the utilities were searched (`utility_only`), fixing the probability tables to their true values. The claims about the *joint* recovery of probabilities, and the dominance of utilities over probabilities (Section 5.9), therefore come almost entirely from the small *bypass* and have not been confirmed on *NHLy*.

The hyperparameters were tuned one at a time. The stopping threshold, the population size and the utility temperature were each fixed on a single preliminary configuration and then held constant across the whole grid; their interactions were not explored, and a value that is good in isolation need not be optimal in every combination. The `top90` criterion, for instance, was calibrated on the `binary` loss and only *assumed* to transfer to the others.

Both diagrams use discrete variables, and *bypass* reduces to two binary decisions. The pipeline was not tested on continuous variables or on decisions with many alternatives, where the decoding and the loss landscape may behave differently. Some findings are, moreover, explicitly network-specific: the effect of the utility temperature (Section 5.6.3) and the dominance of the utilities (Section 5.9) were observed on *bypass* and, as noted there, could change on a diagram with a different geometry.

Because of the degeneracy, the utilities are identified only up to the decisions they in-

duce, never as values. The greedy rule selection of Section 5.8 is an *oracle* that scores candidate rules against the ground-truth policy, so it bounds what an informed elicitation could achieve rather than providing a usable selection rule. The same reliance on the ground truth affects the evaluation itself: accuracy is measured against the *whole* rule base, a signal that would not be available in a real elicitation.

Finally, cost is reported as process CPU seconds of this particular implementation and hardware, so the optimizer trade-offs are implementation dependent; a more optimized version of the expensive optimizers could shift them. As it stands, the most accurate optimizers (KEDA, EGNA) are too costly for the large network, which caps the scale actually reached.

6.3 Future work

This study demonstrates that recovering an Influence Diagram from decision rules is feasible under controlled conditions using two specific networks. Consequently, these findings open several paths for future research. The following sections outline potential extensions, ranging from immediate applications to broader research directions. For each, we highlight the current contributions, limitations, and requirements for further investigation.

6.3.1 Active rule selection.

A key practical finding is that a few highly informative rules are significantly more valuable than many random ones. Specifically, decisive scenarios (those with the largest expected-utility gap) seem to be the most beneficial to elicit (Sections 5.7 and 5.8). Although this study definitively demonstrates the substantial performance spread between a high-quality and a poor rule set, the hypothesis that this optimal sequence can be found simply by selecting the most decisive rules (those with the largest expected-utility gap) must be rigorously proven or refuted. Consequently, future work must validate this specific approach or explore alternative methodologies to identify these optimal sequences. One potential direction is to develop an independent scoring metric.

For example, selecting the rule that maximizes expected information gain (the one that most reduces the spread of the surviving population) would transform the recovery process into an active-elicitation loop. This directly addresses the elicitation bottleneck, as expert effort is measured by the number of rules provided rather than the accuracy achieved.

6.3.2 Exploiting the degenerate population instead of fighting it.

Although degeneracy has been treated as a primary obstacle, the search process generates a population of satisfying models that agree on the training rules but diverge elsewhere (the gap between the best and worst individuals shown in Sections 5.5 and 5.5.3). This study notes this diversity but ultimately discards it, reporting only the parameters of the single best individual or the mean of the satisfiers. However, this diverse population is potentially the most valuable byproduct of the method. Aggregating these models (e.g., through a majority vote on decisions or by characterizing the degenerate subspace) could produce a consensus policy more robust than

any single model.

Furthermore, internal disagreement offers a model-free metric of how effectively the rules constrain the diagram, providing a natural stopping criterion for elicitation. Future research should formally define these aggregation methods, compare them against single-best policies, and analyze the relationship between residual disagreement and true off-rule error.

6.3.3 Scaling the search to realistic diagrams.

This study indicates that the most effective optimizer, KEDA, is also the least scalable. Consequently, for networks like *NHLy*, only less computationally demanding methods like UMDA and EMNA are feasible (Section 5.7). This establishes a theoretical upper bound on quality that cannot yet be achieved at scale (precisely where decision models with thousands of parameters operate, making their recovery valuable). Bridging this scalability gap is a critical priority. Promising approaches include scalable multivariate or surrogate-assisted EDAs, hybrid methods combining EDAs with gradient-based refinement, and a balanced warm-starting strategy for sequential rule additions. Until these techniques can deliver KEDA-level accuracy on large networks, the optimal performance of this method remains proven only for smaller diagrams.

6.3.4 From synthetic rules to real elicitation.

The current methodology relies entirely on synthetic rules derived from a known ground truth, assuming complete accuracy. While this cleanly isolates the recovery mechanism, it represents a significant simplification. In practice, human experts provide rules that can be inconsistent, correlated, or erroneous. To address this, future models must explicitly account for rule reliability. This could involve assigning weights to rules, penalizing mutual inconsistencies during the search, and expanding the framework to handle continuous variables. Testing the pipeline with elicited expert rules is essential to confirm the real-world applicability of these findings.

6.3.5 Understanding identifiability, not only measuring it.

Underlying these practical challenges is a fundamental question addressed here only empirically: which specific rules determine which parameters. While degeneracy is frequently observed, its mathematical structure remains uncharacterized. Developing a theoretical framework for identifiability (defining the exact conditions under which a rule set fully determines a policy) would transform the current practical approach into a theoretically grounded methodology. This represents the most profound open question, linking the empirical results of Chapter 5 to the fundamental conditions required for successful decision model recovery.

Bibliography

[1] BayesFusion, LLC. GeNIe Modeler and SMILE Engine. https://www.bayesfusion.com, 2024.

[2] Concha Bielza, Manuel Gómez, Sixto Ríos-Insua, and Juan A. Fernández del Pozo. Structural, elicitation and computational issues faced when solving complex decision making problems with influence diagrams. *Computers & Operations Research*, 27(7–8):725–740, 2000.

[3] Concha Bielza, Manuel Gómez, and Prakash P. Shenoy. Modeling challenges with influence diagrams: Constructing probability and utility models. *Decision Support Systems*, 49(4):354–364, 2010.

[4] Concha Bielza, Manuel Gómez, and Prakash P. Shenoy. A review of representation issues and modeling challenges with influence diagrams. *Omega*, 39(3):227–241, 2011.

[5] H. Boot, B. G. Taal, and Peter J.F. Lucas. Computer-based decision support in the management of primary gastric non-Hodgkin lymphoma. *Methods of Information in Medicine*, 37(3):206–219, 1998. PMID: 9787619.

[6] Craig Boutilier, Relu Patrascu, Pascal Poupart, and Dale Schuurmans. Constraint-based optimization and utility elicitation using the minimax decision criterion. *Artificial Intelligence*, 170(8–9):686–713, 2006.

[7] Darius Braziunas and Craig Boutilier. Minimax regret based elicitation of generalized additive utilities. In *Proceedings of the 23rd Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 25–32, 2007.

[8] Sabarathinam Chockalingam and Clara Maathuis. Influence diagrams in cyber security: Conceptualization and potential applications. In *Proceedings of the 22nd European Conference on Cyber Warfare and Security (ECCWS)*, 2023.

[9] Dawn Dowding and Carl Thompson. Evidence-based decisions: The role of decision analysis. In *Clinical Decision Making and Judgement in Nursing*, chapter 11. Churchill Livingstone, 2004.

[10] Kazuo J. Ezawa and Narendra K. Gupta. Learning influence diagram from data. In *Proceedings of the Sixth International Workshop on Artificial Intelligence and Statistics (AISTATS)*, PMLR R1, pages 183–190, 1997.

[11] Juan A. Fernández del Pozo, Concha Bielza, and Peter J.F. Lucas. Constructing explanatory prognostic profiles from conditional probability tables by cluster-

KBM2L analysis. In *13th Biennial European Meeting of the Society for Medical Decision Making (SMDM)*, 2010.

[12] Manuel Gómez, Concha Bielza, Juan A. Fernández del Pozo, and Sixto Ríos-Insua. A graphical decision-theoretic model for neonatal jaundice. *Medical Decision Making*, 27(3):250–265, 2007.

[13] Javier Gómez Castellanos and Hugo Terashima-Marín. Solving mixed influence diagrams by reinforcement learning. In *MICAI 2024*, volume 14765 of *LNAI*, pages 209–222, 2024.

[14] Daniel Henríquez Quesada. Recovery of influence diagram parameters by means of genetic algorithms. Master's thesis, Universidad Politécnica de Madrid, 2024.

[15] Ronald A. Howard and James E. Matheson. Influence diagrams. *Decision Analysis*, 2(3):127–143, 2005.

[16] Paul Kailiponi. Analyzing evacuation decisions using multi-attribute utility theory (MAUT). *Procedia Engineering*, 3:163–174, 2010.

[17] Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, 2009.

[18] Pedro Larrañaga and Concha Bielza. Estimation of distribution algorithms in machine learning: A survey. *IEEE Transactions on Evolutionary Computation*, 28(5):1301–1321, 2024.

[19] Pedro Larrañaga and José A. Lozano, editors. *Estimation of Distribution Algorithms: A New Tool for Evolutionary Computation*. Kluwer Academic Publishers, 2002.

[20] Márcio Machado and Mohamed-Habib Karray. Maintenance optimization approaches for condition based maintenance: A review and analysis. In *INCOSE International Symposium*, volume 26, pages 1073–1088, 2016.

[21] Aliakbar Nafar, Kristen Brent Venable, Zijun Cui, and Parisa Kordjamshidi. Extracting probabilistic knowledge from large language models for Bayesian network parameterization. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, PMLR, 2025. arXiv:2505.15918.

[22] Yinghui Pan, Mengen Zhou, et al. Variational auto-encoder based solutions to interactive dynamic influence diagrams. *arXiv preprint arXiv:2409.19965*, 2024.

[23] Yinghui Pan, Mengen Zhou, Biyang Ma, Yifeng Zeng, Yew-Soon Ong, and Guoquan Liu. Improved decisions for unknown behaviours in interactive dynamic influence diagrams. *Artificial Intelligence Review*, 58:361, 2025.

[24] David Ríos Insua, Concha Bielza, and Alfonso Mateos. *Fundamentos de los Sistemas de Ayuda a la Decisión*. Ra-Ma, 2005.

[25] Sixto Ríos-Insua, Alfonso Mateos, and Antonio Jiménez-Martín. La teoría de la utilidad para modelos de preferencias en decisión multiatributo. In *Toma de Decisiones con Criterios Múltiples*, Monografías. Tirant lo Blanch, 2002.

[26] Ahti Salo, Juho Andelmin, and Fabricio Oliveira. Decision programming for mixed-integer multi-stage optimization under uncertainty. *European Journal of Operational Research*, 304(3):1039–1052, 2022.

[27] Ross D. Shachter. Evaluating influence diagrams. *Operations Research*, 34(6):871–882, 1986.

[28] Abida Shahzad. *Multi-Criteria Decision Making Using Bayesian Networks*. PhD thesis, Monash University, Australia, 2022.

[29] Prakash P. Shenoy. Decision trees and influence diagrams. In *Encyclopedia of Life Support Systems (EOLSS): Optimization and Operations Research*, volume 4, pages 280–298. EOLSS Publishers / UNESCO, 2009.

[30] Vicente P. Soloviev, Pedro Larrañaga, and Concha Bielza. EDAspy: An extensible Python package for estimation of distribution algorithms. *Neurocomputing*, 598:128043, 2024.

[31] John von Neumann and Oskar Morgenstern. *Theory of Games and Economic Behavior*. Princeton University Press, Princeton, NJ, 1944.